

PARKING SYSTEM – DISTRIBUTED MICROSERVICES SOFTWARE QUALITY ASSURANCE PLAN

SQAP-G5-V1.0

JULY 18, 2026

University of the Armed Forces – ESPE

Software Engineering - Software Quality Assurance

PARKING SYSTEM – DISTRIBUTED MICROSERVICES

SOFTWARE QUALITY ASSURANCE PLAN

SQAP-G5-V1.0

JULY 18, 2026

SQA Plan Approvals:

Mateo Iza **22/07/2026**

SQA Manager **Date**

Mateo Criollo **22/07/2026**

Project Manager **Date**

Ing. Diego Gamboa **22/07/2026**

Program Manager **Date**

PREFACE

This document contains the Software Quality Assurance (SQA) Plan for the Parking System — Distributed Microservices. The SQA activities described in this plan are consistent with the Parking System — Distributed Microservices Software Development Plan (*or Project Management Plan*) and other project planning documents (README.md, Postman collections, docker-compose.yml). This document has been tailored from the SQA Plan Template, TM-SQA-01, v2.0.

The team #4 of the Software Quality Assurance (ESPE) course assumes responsibility for this document and updates it, as required, to meet the needs of Parking System — Distributed Microservices. Users of this document may report deficiencies or corrections using the Document Change Request found at the end of the document. Updates to this document will be performed, at least annually, in accordance with the proposed work rubric for this third part.

RECORD OF CHANGES

***A** - ADDED **M** - MODIFIED **D** - DELETED

VERSION NUMBER	DATE	NUMBER OF FIGURE, TABLE OR PARAGRAPH	A* M D	TITLE OR BRIEF DESCRIPTION	CHANGE REQUEST NUMBER
1.0	July 18, 2026	Full document	A	Initial SQAP Release	---
1.1	July 20, 2026	Sections 3, 5.1, 6, 8	M	Test strategy, entry/exit criteria, metrics, and toolchain added (P2)	---
1.2	July 20, 2026	Section 7; Tests/ artifacts	A	Dynamic testing executed: defect register, traceability matrix, and manual E2E scripts added (P4)	---
1.3	July 21, 2026	Sections 6.7, 6.8, 7.1, 7.3, 7.4; Appendix C	A	Representative test cases, traceability matrix, defect evidence figures, tool/facilities evaluation, Test Summary Report appendix, and closure verdict against Section 6.4 exit criteria added (P4)	---
1.4	July 21, 2026	Sections 8, Appendix C.7-C.8	A	JaCoCo (backend) and Vitest v8 (frontend) coverage actually executed; X-05 updated with real numbers; new coverage-findings appendix documenting the gap between Section 3.3's unit-test claims and measured business-logic coverage added (P4)	---

--	--	--	--	--	--

TABLE OF CONTENTS

Section	Page
SECTION 1. PURPOSE	1
1.1 Scope	1
1.2 Identification.....	1
1.3 System Overview.....	2
1.4 Document Overview.....	3
1.5 Relationship to Other Plans	4
1.6 Reference Documents.....	5
SECTION 2. MANAGEMENT	1
2.1 Organization	1
2.2 Resources.....	2
2.2.1 Facilities and Equipment.....	2
2.2.2 Personnel.....	2
SECTION 3. SQA TASKS	1
3.1 Task: Review Software Products.....	1
3.2 Task: Evaluate Software Tools.....	1
3.3 Task: Evaluate Software Implementation and Unit Testing Process.....	2
3.4 Task: Evaluate Unit Integration and Testing, CI Qualification Testing, and System Qualification Testing Processes.....	3
3.5 Task: Evaluate the Corrective Action Process.....	3
SECTION 4. DOCUMENTATION.....	1
SECTION 5. STANDARDS, PRACTICES, CONVENTIONS AND METRICS	1
5.1 Metrics	1
5.1.1 Risk-Based Test Priority	2
SECTION 6. TEST	1
6.1 Selected Test Types	1
6.2 Test Environment and Data	2
6.3 Entry Criteria	2
6.4 Exit and Decision Criteria	3
6.5 Suspension and Resumption Criteria.....	4
6.6 Test Case Template	4
6.7 Representative Test Cases (Executed Sample).....	5
6.8 Traceability Matrix (Requirement ↔ Test Case)	6
SECTION 7. SQA PROBLEM REPORTING AND RESOLUTION	10
7.1 Process Audit Report	10
7.1.1 Submittal and Disposition of Process Audit Report	12
7.1.2 Escalation Procedure for Resolution of Non-Concurrence on Process Audit Report.....	12
7.2 Recording Problems in Software Code or Documentation.....	13
7.3 Software Tool Evaluation Report	13
7.4 Facilities Evaluation Report	13

SECTION 8. TOOLS, TECHNIQUES, AND METHODOLOGIES	1
SECTION 9. CODE CONTROL	1
SECTION 10. MEDIA CONTROL.....	1
SECTION 11. SUPPLIER CONTROL	1
SECTION 12. RECORDS COLLECTION, MAINTENANCE AND RETENTION	1
SECTION 13. TRAINING	1
SECTION 14. RISK MANAGEMENT.....	1
APPENDIX A. LIST OF ACRONYMS.....	1
APPENDIX B. GENERAL SOFTWARE ENGINEERING PROCESS AUDIT CHECKLISTS	1
APPENDIX C. TEST SUMMARY REPORT (INFORME DE CIERRE).....	1
C.1 Executive Summary.....	1
C.2 Results by Module (Backend API)	1
C.3 Results by Area (Frontend, CI-07).....	1
C.4 Defect Density	2
C.5 Coverage and Known Gaps	2
C.6 Conclusion of This Run	2
C.7 Closure Verdict Against Exit Criteria (Section 6.4)	3
C.8 Coverage Findings: Claimed vs. Actual Unit Coverage (Before / After).....	4

LIST OF FIGURES

Figure	Page
Figure 1-1. Parking System — Distributed Microservices Software.....	4
Figure 2-1. Parking System — Distributed Microservices Organization	1
Figure 7-4. DEF-01 — Login error message missing accented characters.....	11
Figure 7-5. DEF-02 — Unauthorized "from" redirect lands on Access Denied after login	11
Figure 7-6. DEF-03 — "My Profile" renders with blank personal-data fields	12
Figure 7-7. DEF-04 — Ticket-cancellation modal loses focus and self-closes	12
Figure 7-2. Software Tool Evaluation.....	15
Figure 7-3. Project Facilities Evaluation.....	17
Figure B-1. Project Planning Process Audit Checklist	3
Figure B-2. Project Tracking and Oversight Process Audit Checklist.....	4
Figure B-3. System Requirements Analysis Process Audit Checklist.....	5
Figure B-4. System Design Process Audit Checklist.....	6
Figure B-5. Software Requirements Analysis Process Audit Checklist	7
Figure B-6. Software Design Process Audit Checklist	8
Figure B-7. Software Implementation and Unit Testing Process Audit Checklist	10
Figure B-8. Unit Integration and Testing Process Audit Checklist	11
Figure B-9. CI Integration Testing and System Qualification Process Audit Checklist.....	13
Figure B-10. End-Item Delivery Process Audit Checklist.....	17
Figure B-11. Software Corrective Action Process Audit Checklist.....	19

Figure B-12. Media Certification Process Audit Checklist	22
Figure B-13. Non-Deliverable Software Certification Process Audit Checklist	24
Figure B-14. Storage and Handling Process Audit Checklist	25
Figure B-15. Subcontractor Control Process Audit Checklist	26
Figure B-16. Deviations and Waivers Process Audit Checklist	28
Figure B-17. Software Configuration Management Process Audit Checklist	29
Figure B-18. Software Development Library Control Process Audit Checklist.....	33
Figure B-19. Non-Developmental Software Process Audit Checklist.....	35

LIST OF TABLES

Table	Page
Table 1-1. Software Lifecycle Activities	2
Table 1-1a. SCOPE MATRIX	2
Table 1-2. CI Nomenclature/Identification.....	3
Table 3-1. Product and Artifact Review Criteria	1
Table 3-2. Software Tool Acceptance Criteria	2
Table 4-1. Deliverable Software Products	1
Table 4-2. Non-deliverable Software Products.....	1
TABLE 13-1. SQA TRAINING MATRIX (BASE TEMPLATE REFERENCE — NOT EXECUTED)...	1
Table 14-1. Product Risks.....	1
Table 14-2. Project Risks.....	2

SECTION 1. PURPOSE

The purpose of this plan is to define the Parking System — Distributed Microservices Software Quality Assurance (SQA) organization, SQA tasks and responsibilities; provide reference documents and guidelines to perform the SQA activities; provide the standards, practices and conventions used in carrying out SQA activities; and provide the tools, techniques, and methodologies to support SQA activities, and SQA reporting.

1.1 SCOPE

This plan establishes the SQA activities performed throughout the life cycle of the Parking System — Distributed Microservices.

Team 5 acts as an external quality consultancy, independent from the original development team that built the Parking System in a previous academic term. This independence satisfies the IEEE 730 requirement that the SQA function maintain a reporting channel free of conflicts of interest with the parties responsible for building the software: the SQA team has no involvement in, and no stake in, the original design, coding, or delivery decisions being audited.

The goal of the SQA program is to verify that all software examined meets its technical and functional requirements. The SQA procedures defined herein are used to examine the in-scope deliverable software to determine compliance with technical and performance requirements.

Table 1-1 shows the software life cycle activities of the Configuration Items (CIs), as identified by the Institute of Electrical and Electronics Engineers (IEEE)/Electronic Industries Association (EIA) Standard 12207 Series, Software Life Cycle Process, reference (c), to which this SQA Plan applies.

1.2 IDENTIFICATION

Table 1-2 shows the CIs to which this plan applies.

Listed below is a brief description of each of the CIs comprising the Parking System — Distributed Microservices, originally developed in a previous academic term and audited by Team 5:

- a. CI-01 – Users microservice (ms-users): manages authentication, users, roles and accounts; issues and validates JWT tokens for the entire system.
- b. CI-02 – Microservice for zones (ms-zones): manages zones and individual parking spaces, including real-time availability queries.
- c. CI-03 – Vehicle microservice (ms-vehicles): manages vehicle registration and search, including search by license plate.
- d. CI-04 – Assignment Microservice (ms-assignments): links owners/users with their vehicles and manages the assignment of vehicles to parking spaces.
- e. CI-05 – Ticket microservice (ms-tickets): manages the lifecycle of parking tickets (entry/exit); currently does not have automated test coverage.
- f. CI-06 – Audit microservice (ms-audit): consumes domain events published by the other microservices via RabbitMQ (audit_exchange) and stores them for traceability; currently, it does not have automated test coverage.

- g. CI-07 – Web frontend (frontend): React 18 + TypeScript single-page application built with Vite; provides authentication/RBAC, administration, and customer-facing (portal) interfaces, consuming the six microservices through the API Gateway. Added to the project after the initial scope definition; currently has no automated test coverage (no test runner declared in package.json).

1.3 SYSTEM OVERVIEW

The Parking System — Distributed Microservices consists of a backend platform composed of independently deployable microservices that manage parking lot operations: user authentication and roles, parking zone and space administration, vehicle registration, vehicle-to-space assignments, ticket lifecycle management, and audit-trail event persistence. The system is intended to support the day-to-day operation of a parking facility, exposing REST APIs consumed by client applications through a single API Gateway entry point. As of this audit, the system also includes a web frontend (React 18 / TypeScript single-page application) that serves as the primary client, providing authentication (RBAC), administration, and customer-facing (portal) interfaces on top of the six microservices. The system includes 7 subsystems (Configuration Items) within the system, described in Table 1-2. Figure 1-1 identifies the CIs within the system and highlights those to which this SQA Plan applies.

TABLE 1-1. SOFTWARE LIFECYCLE ACTIVITIES

SOFTWARE LIFECYCLE ACTIVITY
Software Development Environment
Software Requirements Analysis
Software Implementation and Unit Testing
Unit Integration and Testing
CI Qualification Testing
CI/Hardware Configuration Item (HWCI) Integration and Testing

TABLE 1-1A. SCOPE MATRIX

MODULE	TECHNOLOGY	IN/OUT OF SCOPE	PRIORITY
User	Spring Boot 3.5 / Java 25	IN SCOPE	Medium
Zones	Spring Boot 3.5 / Java 25	IN SCOPE	Medium
Vehicle	NestJS 11 + TypeORM	IN SCOPE	Medium
Assignments	Spring Boot 3.5 / Java 25	IN SCOPE	Medium
Ticket	Spring Boot 3.5 / Java 25	IN SCOPE	High

Audit	NestJS 11 + TypeORM	IN SCOPE	High
gateway (Kong)	Kong 3.7 (declarative)	OUT OF SCOPE	N/A
monitoreoDashboard	HTML/CSS/JS + Tailwind CDN	OUT OF SCOPE	N/A
Frontend	React 18 + TypeScript + Vite	IN SCOPE	High

TABLE 1-2. CI NOMENCLATURE/IDENTIFICATION

NOMENCLATURE	ACRONYM	CI NUMBER
User Microservice	ms-usuarios	CI-01
Zones Microservice	ms-zones	CI-02
Vehicle Microservice	ms-vehicles	CI-03
Assignments Microservice	ms-assignments	CI-04
Ticket Microservice	ms-tickets	CI-05
Audit Microservice	ms-audit	CI-06
Web Frontend	frontend	CI-07

1.4 DOCUMENT OVERVIEW

This document identifies the organizations and procedures to be used to perform activities related to the Parking System — Distributed Microservices SQA program as specified in IEEE Std 730-1998, IEEE Standard for Software Quality Assurance Plans, reference (c) and IEEE Std 730.1-1995, IEEE Guide for SQA Planning, reference (d).

Section 1 identifies the system to which this SQA Plan applies; provides an overview of the system and its software functions; summarizes the purpose and contents of the SQA Plan; and describes the relationship of the SQA Plan to other management plans and lists all documents referenced in this SQA Plan.

Section 2 describes each major element of the organization that influences the quality of the software.

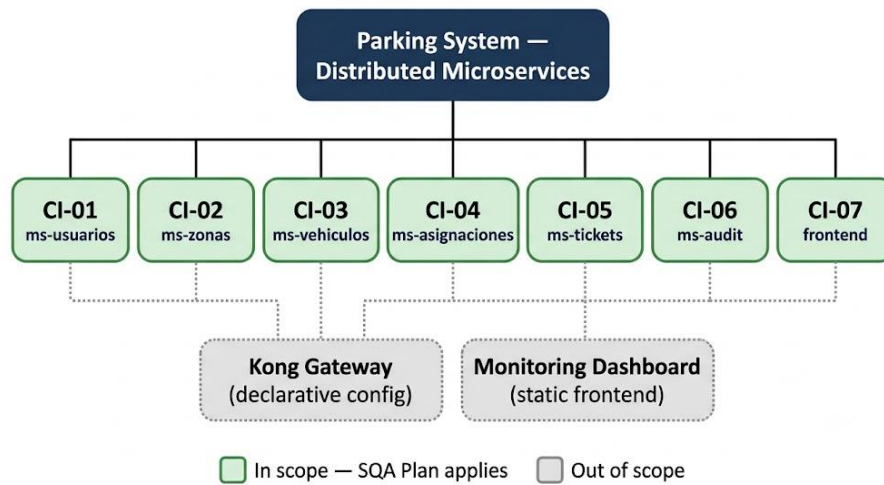


Figure 1-1. Parking System — Distributed Microservices Software

Section 3 describes the various SQA tasks

Section 4 lists the baseline documents produced and maintained by the project.

Section 5 identifies the standards, practices and conventions.

Section 6 describes SQA involvement in testing.

Section 7 describes problem reporting and corrective action.

Section 8 describes SQA tools, techniques, and methodologies.

Section 9 describes the configuration management tool used for code control.

Section 10 describes SQA evaluation of media control.

Section 11 describes control of supplier software.

Section 12 describes SQA procedures for record collection, maintenance, and retention.

Section 13 describes SQA training requirements.

Section 14 describes SQA review of the Risk Management process.

Appendix A provides a list of acronyms.

1.5 RELATIONSHIP TO OTHER PLANS

The Parking System — Distributed Microservices does not have a formal, independent Software Development Plan (SDP) or Software Configuration Management Plan (SCM), as would be expected in a project developed under a formal project management framework. Since this Software Quality Assurance (SQA) covers an audit of an existing system, rather than a complete development lifecycle, the SQA assessment in this document is based on the project's existing foundational documentation, primarily the README.md file (reference (e)), which describes the system's architecture, configuration, and operating conventions. Software configuration management is performed using Git/GitHub, as described in more detail in Section 9, Code Control.

1.6 REFERENCE DOCUMENTS

This section lists the documents referenced in this SQA Plan.

- a. IEEE Std 730-1998, IEEE Standard for Software Quality Assurance Plan, June 1998.
- b. IEEE Std 730.1-1995, IEEE Guide for Software Quality Assurance Planning, December 1995.
- c. IEEE/EIA Standard 12207 Series, Standard for Information Technology – Software Life Cycle Processes, March 1998.
- d. Software Quality Assurance Plan Template, TM-SQA-01, v2.0.
- e. README.md, Parking System — Distributed Microservices project repository.
- f. TICKET-auditoria.md, audit system specification, project repository (docs/).
- g. Postman collections (usuarios, zonas, vehiculos, asignaciones, gateway environment), project repository (postman/).
- h. docker-compose.yml, project repository — PostgreSQL and RabbitMQ infrastructure definition.
- i. kong.yml, project repository (gateway/) — API Gateway declarative configuration.

SECTION 2. MANAGEMENT

This section describes each major element of the organization that influences the quality of the software.

2.1 ORGANIZATION

Good software practices require a certain degree of independence for the Software Quality Assurance (SQA) team. This independence is a key strength for SQA; that is, SQA is free to report any product quality issues directly to a higher level than the project level. While in practice this rarely happens, as almost all problems are successfully addressed at the project level, the fact that the SQA team can go beyond that level allows it to keep many of these issues contained there.

Figure 2-1 shows the SQA organization in relation to the project organization.



Figure 2-1. Parking System — Distributed Microservices Organization

The Software Quality Assurance (SQA) department is responsible for ensuring compliance with SQA requirements, as described in this SQA Plan. The SQA organization ensures the quality of the audited software and its documentation, as well as the engineering processes used in its production.

The functional groups that influence and control software quality in this audit are described below.

Program Management / Sponsor (Instructor Diego Gamboa) is responsible for the following items:

1. Approving the scope and overall approach of this SQA Plan.
2. Reviewing and approving the final SQAP document.
3. Resolve and follow up on quality or rating concerns raised by the SQA team.
4. Confirm that the SQA team is independent of the original development team, thus meeting the independence requirement of the IEEE 730 standard described in Section 1.1.

Project / SQA Leadership (Mateo Criollo) is responsible for the following items:

1. Definition of scope, risk management, and overall organization of Software Quality Assurance (SQA) (sections 1, 2, and 14).
2. Review and approve the work done by the group before final consolidation.
3. Consolidation, formatting, and finalization of the SQAP document.
4. Resolution and follow-up of quality issues identified during the audit.

Test Strategy and Tooling (Eduardo Garcia) is responsible for the following items:

1. Define the types of tests applicable to this audit, the entry/exit criteria, and the metrics to be collected (sections 3, 5, 6, and 8).
2. Selection and justification of the toolchain for static analysis, unit testing, and functional testing.
3. Provide the standardized test case template used by Dynamic Testing and Defect Management.

Static Analysis and Technical Debt (Josue Guallichico) is responsible for the following items:

1. Perform a static analysis of the audited system and obtain the "before" and "after" reports (Sections 4 and 9).
2. Refactoring critical/blocking problems without altering functional behavior.
3. Document code control practices and modules marked as Not Applicable (sections 10 to 13).

Dynamic Testing and Defect Management (Mateo Iza) is responsible for the following items:

1. Design and execute unit and functional test cases for the microservices and the web frontend included in the project scope.
2. Defect lifecycle management: detection, notification and verification (Section 7).
3. Prepare the traceability matrix and the Test Summary Report.

2.2 RESOURCES

2.2.1 Facilities and Equipment

The SQA team will have access to the facilities and equipment described in reference (e), including local development workstations for each member, with JDK 25, Node.js 20+, and Docker Desktop (for the PostgreSQL and RabbitMQ containers described in the project's docker-compose.yml file), and a current version of Google Chrome (or equivalent) for exercising the React frontend. SQA will also have access to Git/GitHub, Postman, SonarQube/SonarCloud accounts, and ESLint for static analysis. These computing resources support SQA's process and product evaluation and auditing functions throughout the period.

2.2.2 Personnel

The System Quality Assurance (SQA) work for this project is distributed among the four members of Team 5, who work part-time during Partial 3, in parallel with their studies in other subjects, following the task sequence defined in Section 2.1.

The Software Quality Assurance Manager must know and apply the standards and guidelines listed in Section 1.6. In addition, the Software Quality Assurance Manager must be familiar with software quality concepts, software development activities, the architecture of the audited system, and structured analysis, design, coding, and testing practices relevant to the Java/Spring Boot, TypeScript/NestJS, and React/TypeScript (frontend) platforms used in the Parking System.

SECTION 3. SQA TASKS

This SQA Plan tailors Section 3 to the tasks that are meaningful for an academic audit of an already-built system (the SUT), rather than the full software-development lifecycle assumed by the base template. SQA for the Parking System — Distributed Microservices audit performs, to their full extent, the tasks below: reviewing the software products that support test execution and traceability (3.1), evaluating the tools used for static analysis and dynamic testing (3.2), evaluating unit testing (3.10), evaluating integration, system, and end-to-end testing (3.11), and running the corrective-action process for defects found during the audit (3.13).

Tasks 3.3–3.9, 3.12, and 3.14–3.25 of the base template — which assume a full requirements/design lifecycle under audit, physical media, subcontractors, and a formal Software Development Library — are marked Not Applicable to this academic engagement, since this audit covers an already-built system rather than its original development. The concerns they cover are addressed instead in Section 1 (scope), Section 4 and Section 9 (documentation and code control), and Section 14 (risk).

3.1 TASK: REVIEW SOFTWARE PRODUCTS

SQA reviews, through risk-directed sampling, the products that support test execution and traceability for this audit. Each review records the product, version/commit, reviewer, date, criterion applied, findings, and disposition.

TABLE 3-1. PRODUCT AND ARTIFACT REVIEW CRITERIA

Product	Review Criterion	Minimum Evidence
Code and configuration	Compiles; naming conventions followed; no versioned secrets; endpoints and environment variables are consistent	Commit hash, build output, and static-analysis findings
API contracts	Routes, DTOs, HTTP status codes, authentication, and examples are consistent	docs/contrato-api-microservicios.md + Postman collection
Test cases	Traceable, repeatable, with test data and verifiable expected results	Requirement/case ID and peer review
Test scripts	Isolated under Tests/AreadePruebas, deterministic, and documented	Exact command + result
Reports	Date, version, reconciled totals, and legible screenshots	static-before/after, coverage, and test-summary reports

3.2 TASK: EVALUATE SOFTWARE TOOLS

A tool is accepted for use in this audit when it is compatible with the SUT's actual languages and versions, produces exportable evidence, can be executed reproducibly, and covers a rubric need without unnecessarily duplicating another tool.

TABLE 3-2. SOFTWARE TOOL ACCEPTANCE CRITERIA

Criterion	Weight	Acceptance Rule
Technical compatibility	25%	Runs against Java 25, Node 20+, and the current repository
Coverage of the need	25%	Detects/measures the required attribute and distinguishes between modules
Reproducibility	20%	Documented command with a repeatable result on another machine
Traceability / evidence	15%	Exports a capturable report or output with date/version
Cost and adoption	10%	Viable for academic use with a reasonable learning curve
Integration	5%	Integrates with Maven, npm, Git, or the API collection

Minimum acceptance score: 75/100, with no failure on the compatibility criterion. If a tool modifies code by default, the audit run must be configured in read-only mode — for example, ESLint must not be run with --fix during the 'before' capture.

3.3 TASK: EVALUATE SOFTWARE IMPLEMENTATION AND UNIT TESTING PROCESS

SQA verifies that critical units have positive, negative, and boundary-value tests; that external dependencies are mocked where appropriate; and that coverage results come from the same commit under audit.

Module	Priority Units / Risks	Technique
usuarios	JWT, password, national ID, role-based authorization	JUnit 5 + Mockito + spring-security-test
zonas	Availability, state changes, space rules	JUnit 5 + Mockito
vehiculos	License-plate/type validation, search, service, and controller	Jest + repository mocks
asignaciones	Duplicate detection, validity window, external catalogs, states	JUnit 5 + Mockito
tickets	Fare calculation, code generation, compatibility, state transitions	JUnit 5 + Mockito
ms-audit	DTOs, consumer, persistence, filters, and JWT	Jest + AMQP/TypeORM mocks

frontend (CI-07)	RBAC guards and route protection (rbac.ts, guards.tsx), formatting/validation utilities (format.ts)	Vitest + React Testing Library
------------------	---	--------------------------------

The frontend (CI-07) was incorporated into this task after the initial scope definition (Risks RP-05 and RQ-07, Section 14); its unit-testing surface is intentionally narrow — the RBAC/authorization logic and the formatting/validation utilities that gate access and data correctness in the UI — since full component-level testing of every screen is out of scope for this audit.

3.4 TASK: EVALUATE UNIT INTEGRATION AND TESTING, CI QUALIFICATION TESTING, AND SYSTEM QUALIFICATION TESTING PROCESSES

Integration tests validate real boundaries: HTTP, PostgreSQL persistence, RabbitMQ messaging, and JWT-based security. System tests traverse Kong and, since the frontend is in scope (CI-07), the web UI as well, verifying the visible or queryable result, including audit side effects.

Authentication and authorization: login in usuarios, JWT issuance, role-based access granted/denied, validated at both the Kong/service level and in the frontend's route guards.

Parking entry: existing vehicle, valid assignment, available space, ticket creation, and audit event.

Exit and payment: active ticket, fare calculation, state transition, logical release of the resource, and audit event.

Asynchronous traceability: a write operation, publication to audit_exchange, consumption on audit_queue, and a subsequent query in ms-audit.

Controlled resilience: invalid token, unavailable dependency, non-existent data, duplicates, and malformed requests.

Frontend critical flows (system/end-to-end, CI-07): login and session handling, RBAC-gated navigation across admin/portal screens, and ticket issuance/cancellation exercised through the real UI (Section 6 lists the corresponding manual scripts).

3.5 TASK: EVALUATE THE CORRECTIVE ACTION PROCESS

1. Record the finding with an ID, module, version, severity, priority, preconditions, steps, expected/actual result, and evidence.
2. Triage: Blocker/Critical is handled the same day; High within 24 hours; Medium/Low within the quality sprint.
3. Assign an owner and link the defect to the requirement, test case, and commit or pull request.
4. Fix it on a separate branch, run regression tests, and attach before/after evidence.
5. Close only after Dynamic Testing (P4) verifies the result in the agreed environment; if it reappears, reopen it while keeping the history.

Table 3-3 summarizes how this task set connects to the rest of the SQA team; it complements, at an academic scale, the responsibility matrices of the base template (Task 3.24), which assume a much larger contractor organization.

Role	Responsibility Related to This Task Set	Output
------	---	--------

P1 — SQA Leadership	Confirms scope and risk; consolidates the SQAP	Approved Scope Matrix and final document
P2 — Test Strategy and Tooling	Defines strategy, toolchain, metrics, criteria, and test-case format	Sections 3, 5, 6, and 8
P3 — Static Analysis	Runs static analysis with the toolchain defined here	Before/after reports
P4 — Dynamic Testing	Designs/executes tests and computes the metrics	Test cases, defects, and Test Summary Report

SECTION 4. DOCUMENTATION

This SQA Plan tailors Section 4 to an already-built academic SUT. The base template assumes deliverable documentation produced under MIL-STD-498 Data Item Descriptions (DIDs); because the Parking System — Distributed Microservices was developed in a previous academic term without a DID-based documentation set, Section 4 instead catalogs the baseline documents that actually exist in the project repository and the quality artifacts produced by this audit. The "standard / guideline" column therefore records the convention actually followed (Markdown, Maven, npm, Docker Compose, Postman, or this SQAP), not a DID reference.

Table 4-1 lists the baseline and audit-deliverable products that describe and support the software; Table 4-2 lists the non-deliverable products maintained internally by the SQA team. Both are versioned in Git (Section 9) and identified by commit hash. For any software document not yet listed, SQA will assist in identifying the convention to be followed in its preparation.

TABLE 4-1. DELIVERABLE SOFTWARE PRODUCTS

NOMENCLATURE	DELIVERABLE DOCUMENTATION	STANDARD / GUIDELINE
README.md	System architecture, setup and operations guide (reference (e))	Markdown; project conventions
docs/TICKET-auditoria.md	Audit subsystem (ms-audit) specification	Markdown; project conventions
postman/ collections (usuarios, zonas, vehiculos, asignaciones, gateway)	Executable API contract with request/response examples	Postman collection format
docker-compose.yml	Infrastructure definition: PostgreSQL 16 and RabbitMQ 3 services	Docker Compose specification
gateway/kong.yml	API Gateway declarative routing and security configuration	Kong 3.7 declarative configuration
Source code and build descriptors per CI-01 to CI-07 (pom.xml, package.json)	Self-documented source and dependency/build manifests	Maven (Java) and npm (TypeScript) conventions
reports/static-before.md and reports/static-after.md	Static-analysis baseline and post-refactor reports (this audit)	SonarCloud metrics model; SQAP Section 5.1

TABLE 4-2. NON-DELIVERABLE SOFTWARE PRODUCTS

DOCUMENT TITLE

Tests/ suite and Tests/README.md (exact reproduction commands for static analysis, tests, and coverage)
reports/ — coverage and test-summary reports
traceability-matrix.csv — requirement-to-test-case traceability
defects.md — defect board export (Trello/Jira format)
.github/workflows/sonarcloud-*.yml — CI pipelines that regenerate analysis and coverage evidence
SonarCloud "Overall Code" dashboard captures — 14 images (before/after per module), Section 5.1

Document adequacy is confirmed by peer review within the SQA team together with automated verification, rather than by submission to a formal SCM or SEPO organization, which does not exist in this academic engagement. Each baseline document is checked against the review criteria in Section 3.1 (Table 3-1): the product, its version or commit, the reviewer, the date, the criterion applied, the findings, and the disposition are recorded.

Static-analysis reports (reports/static-before.md and reports/static-after.md) are considered adequate only when they are generated from the same audited commit, exported from SonarCloud with date and version, and reconciled against the metrics defined in Section 5.1. Upon approval, each document is committed to the Git repository on the working branch (Section 9) and referenced by its commit hash; superseded versions remain recoverable through the repository history.

SECTION 5. STANDARDS, PRACTICES, CONVENTIONS AND METRICS

This SQA Plan tailors Section 5 to an academic SUT, using the IEEE 730 template structure as its baseline, the test practices of ISO/IEC/IEEE 29119, and the ISO/IEC 25010 product-quality model covered in the course, rather than the MIL-STD-498 / DID-based conventions assumed by the original template. Measurements are documented with a definition, formula, source, frequency, and owner, avoiding figures without a verifiable basis.

Scope	Applicable Convention	Verification
Java/Spring	Project conventions, Maven build, JUnit 5 tests	Build, review, and SonarQube
TypeScript/NestJS	Strict TypeScript, ESLint/Prettier, Jest	Build, lint without auto-fix, and SonarQube
Frontend (React/TS, CI-07)	Strict TypeScript, ESLint, component tests with Vitest + React Testing Library	Build (tsc --noEmit && vite build), lint, and Vitest run
API	Consistent HTTP semantics, valid JSON, authentication/authorization, versioned collection	Postman/Newman and the API contract
Tests	Identifiable, traceable, repeatable case with an expected result	Peer review + traceability matrix
Evidence	Commit, date, command, environment, result, and source file	Closure checklist

5.1 METRICS

The measures below are mandatory for this audit; they replace the generic SQA cost/schedule measurements of the base template with metrics that are directly verifiable against the SUT's test execution.

ID	Metric	Formula	Source	Threshold / Use
M-01	% Execution	Executed cases / planned cases × 100	Execution log	100% of critical cases and ≥95% overall
M-02	% Pass rate	Passed cases / executed cases × 100	JUnit/Jest/Vitest/Newman/manual	≥90% overall; 100% on critical cases
M-03	Requirement coverage	Requirements with ≥1 executed case /	Traceability matrix	100% of critical requirements; ≥95% overall

		requirements in scope × 100		
M-04	Line coverage	Covered lines / measurable lines × 100	JaCoCo/Jest/Vitest	≥70% of critical logic per module
M-05	Branch coverage	Covered branches / measurable branches × 100	JaCoCo/Jest/Vitest	≥60% of critical logic per module
M-06	Defect density	Confirmed defects / module	Defect board	Reported by severity; prioritizes hotspots
M-07	Open defects	Count by severity and module	Defect board	0 Blocker/Critical for exit
M-08	Reopen rate	Reopened defects / closed defects × 100	Defect board	≤10%
M-09	Static quality	New/open issues by severity; debt and duplication	SonarQube + ESLint	0 new Blocker/Critical; no unmitigated High

Measured result for M-09 (Static Quality): the baseline capture (2026-07-21 16:24) reported 205 issues across the 7 modules — 3 bugs, 7 vulnerabilities, and 195 code smells — with 1,160 minutes of accumulated technical debt and ~8.8% average coverage. Following the refactoring commits in Table 9-1, the post-refactor capture (2026-07-21 16:36) reported 30 issues — 2 bugs, 0 vulnerabilities, and 28 code smells — with technical debt reduced to 187 minutes and average coverage raised to ~95.1%. All 7 modules pass the SonarCloud Quality Gate in the after state. One residual BLOCKER-severity finding remains in ms-audit (a test file missing an assertion, typescript:S2699); as it resides in test code rather than application logic, it is tracked as an accepted exception to the M-09/X-06 zero-Blocker threshold and coordinated with Dynamic Testing (P4) for resolution.

Interpretation rule: coverage thresholds apply to measurable critical logic, not to getters, trivial DTOs, generated code, or declarative configuration. Dynamic Testing (P4) must report the numerator, denominator, and exclusions for each figure; high coverage does not offset failing functional cases.

5.1.1 Risk-Based Test Priority

Priority	Modules / Flow	Reason	Depth
High	tickets and ms-audit	The Scope Matrix shows no prior test	Exhaustive unit tests + integration + system

		coverage; direct financial and traceability impact (Risk RP-01)	
High	usuarios / authentication	Controls identity, roles, and JWT for the whole system	Security-focused negative tests + integration
High	asignaciones + entry/exit	Coupling between catalogs and business rules	Integration and end-to-end
High	Frontend critical flows: login, RBAC-gated navigation, ticket issuance/cancellation (CI-07)	Added to scope after the initial definition, with no automated coverage before this audit (Risk RP-05); directly exposed to end users	Unit tests over RBAC/guard/format logic + scripted manual end-to-end
Medium	zonas	Availability affects the core operation	Boundary-value unit tests + API
Medium	vehiculos	Domain entry point and license-plate search	Unit tests + API

SECTION 6. TEST

Test execution follows a pragmatic pyramid: a large base of fast unit tests over business rules, a smaller layer of integration tests over critical boundaries, and a handful of system-level scenarios — run through the API Gateway (Kong) and, since the frontend is in scope (CI-07), through the web UI — that demonstrate the complete flow. Case selection is driven by risk and by the approved scope (Section 1.1), not by assigning an equal number of cases to every module.

6.1 SELECTED TEST TYPES

Type	Applies	Objective and Scope	Responsible
Unit	Yes	Pure rules, validations, services, and state transitions in the 6 microservices, plus RBAC/guard and formatting/validation logic in the frontend	Dynamic Testing (P4); criteria audited by P2
Integration	Yes	HTTP between services, PostgreSQL, RabbitMQ, security, and contracts	P4
Functional/API	Yes	Observable behavior through direct endpoints and through Kong	P4
System / End-to-end	Yes, sampled	Login, entry, audit, exit/payment, ticket cancellation, and role-based access control, exercised through the real frontend UI	P4 + team
Regression	Yes	Re-run cases affected by P3/P4 corrections	P4
Basic security	Yes	Missing/invalid JWT, roles, invalid input, and secrets	P3/P4
Performance/Load	Not formal	Outside the available time and without quantified non-functional requirements; only observed response times are noted	Justified

Usability with real users	Not formal	No user sample available for this academic audit	Justified
Cross-platform compatibility	Not formal	Single academic environment; supported versions are documented instead	Justified

6.2 TEST ENVIRONMENT AND DATA

Controlled environment with JDK 25, Node.js 20+, Docker Desktop, PostgreSQL 16, RabbitMQ 3, Kong 3.7, and a current version of Google Chrome to exercise the React frontend; versions and ports are captured before execution.

JWT keys generated by scripts/setup_jwt_keys.sh; the private key is never included in public evidence.

Reproducible data created with scripts/seed_usuarios_roles.py and scripts/seed_datos.py; each run records whether the database was clean or reused.

Role-based accounts: root, qa.admin, qa.recauda, qa.cliente, and qa.invitado; passwords are treated as lab data and are never copied into public reports.

Every piece of evidence records commit, date/time, operating system, tool/version, command, and result.

6.3 ENTRY CRITERIA

ID	Criterion	Evidence	If Not Met
E-01	Scope approved by P1, including the frontend (CI-07)	Reviewed Scope Matrix, Table 1-1a	Do not start the formal campaign
E-02	Target commit/version identified with no undocumented local changes	Hash + git status	Fix the baseline
E-03	All 6 services and the frontend build, and infrastructure dependencies are available	Build + health checks/ports	Log the risk and suspend the affected layer
E-04	Lab configuration and keys prepared without exposing secrets	Environment checklist	Block security/integration tests
E-05	Test data loaded and recoverable	Seed log + query	Reset the data

E-06	Requirements/features identified and linkable to test cases	Preliminary matrix	Complete the traceability
E-07	Critical cases reviewed, with preconditions and observable results	Peer approval	Return to design
E-08	Tools installed/configured and commands documented	Tests/README.md	Resolve the configuration

6.4 EXIT AND DECISION CRITERIA

ID	Criterion	Metric	Decision if Not Met
X-01	100% of critical cases and $\geq 95\%$ of the total planned cases executed	M-01	Reject/suspend closure
X-02	100% of critical requirements and $\geq 95\%$ of the total have an executed case	M-03	Reject
X-03	100% of critical cases passed and $\geq 90\%$ overall pass rate	M-02	Reject, or accept with a formal exception for non-critical cases
X-04	0 open Blocker/Critical defects; no High-severity security defect without approved mitigation	M-07	Reject
X-05	$\geq 70\%$ line and $\geq 60\%$ branch coverage on critical logic per module	M-04/M-05	Justify the gap and the remediation plan; do not conceal it
X-06	0 new Blocker/Critical static-analysis issues after the refactor	M-09	Reject
X-07	Corrections verified through regression, with a reopen rate $\leq 10\%$	M-08	Keep the campaign open
X-08	Evidence, defects, matrix, and Test Summary Report	Checklist	Do not consolidate the PDF

	reconciled against the same commit		
--	------------------------------------	--	--

Acceptance criterion: the SUT is recommended for academic delivery when criteria X-01 through X-04, X-06, and X-08 are all met. Coverage gaps under X-05 may only be accepted as a documented exception when they do not affect critical paths and an explicit residual risk is recorded. Any open Blocker/Critical defect results in rejection.

6.5 SUSPENSION AND RESUMPTION CRITERIA

Suspend When	Resume When
The build fails on the target commit	The build passes again and the incident log is kept
PostgreSQL, RabbitMQ, Kong, or a required service is unstable	Health checks and a control scenario pass twice
Data is contaminated or not reproducible	The database is restored and the seed is verified
A Blocker defect prevents executing subsequent cases	A verified fix or an approved, documented workaround exists
Evidence does not allow the version or result to be identified	Execution is repeated with complete capture

6.6 TEST CASE TEMPLATE

Field	Content to Complete
ID / Title	TC-[MODULE]-NNN — [verb + condition + result]
Related requirement	REQ-___ / user story / endpoint / business rule
Module and priority	usuarios zonas vehiculos asignaciones tickets ms-audit frontend; High/Medium/Low
Type	Unit Integration Functional/API System/E2E Regression Security
Preconditions	Version, services, role, existing data, and required configuration
Test data	Exact inputs, identifiers, and boundary values; no real secrets
Steps	1. ___ 2. ___ 3. ___
Expected result	Observable, measurable response/code/state/event/change
Actual result	Completed during execution
Status	Not executed Passed Failed Blocked
Evidence / defect	Path or screenshot; DEF-___ if it fails
Executed by and date	Name — YYYY-MM-DD HH:MM — commit

6.7 REPRESENTATIVE TEST CASES (EXECUTED SAMPLE)

The table below reproduces a risk-prioritized sample of the test cases actually designed and executed for this audit, using the template defined in Section 6.6. Passing and failing cases are both included so the sample is representative of the real outcome, not only of the successful runs. The complete set — 40 functional API cases, 39 frontend unit cases, and 26 manual E2E steps across three scripted flows — is maintained in the project repository (Tests/, frontend/src/**/*test.ts(x), postman/) and indexed in the Traceability Matrix (Section 6.8).

ID / Title	Requirement	Module (Priority)	Type	Steps (abridged)	Expected Result	Status	Evidence / Defect
TC-USUARIOS-001 — Login with valid credentials issues a JWT	RF-AUTH-01	usuarios (High)	Functional API (Postman/Newman)	1. POST /api/v1/auth/login with valid root credentials.	HTTP 200; response includes a valid JWT and role claims	Passed	Tests/evidence/newman-result.json
TC-USUARIOS-002 — Login with wrong password is rejected	RF-AUTH-02	usuarios (High)	Functional API (Postman/Newman)	1. POST /api/v1/auth/login with a valid username and an incorrect password.	HTTP 401; no token issued	Passed	Tests/evidence/newman-result.json
TC-TICKETS-001 — Vehicle entry opens an ACTIVE ticket	RF-TCK-01	tickets (High)	Functional API (Postman/Newman)	1. POST /api/v1/tickets/ingreso with a registered plate and an available space.	HTTP 201; ticket returned with estadoTicket=ACTIVO, fare code and entry time	Passed	Tests/evidence/newman-result.json
TC-TICKETS-002 — Duplicate active ticket for the same vehicle is rejected	RF-TCK-02	tickets (High)	Functional API (Postman/Newman)	1. Repeat the entry request for a vehicle that already has an active ticket.	HTTP 409, vehicle already has an active ticket	Passed	Tests/evidence/newman-result.json
TC-FE-GRD-002 — RequirePermission blocks a role without permission	RF-FE-GRD-02	frontend / guards.tsx (High)	Unit/Component (Vitest + RTL)	1. Render a protected route with a user role lacking the required permission.	Route redirects to /403 without revealing that the protected route exists	Passed	frontend/src/auth/guards.test.tsx
TC-FE-E2E-01-06b —	RF-FE-E2E-01	frontend / LoginPag	Scripted manual E2E (browser)	1. While logged out, force	User should land on /portal (rutaInicial),	Failed — landed	DEF-02 (Tests/evidence/DEF

Post-login redirect ignores an unauthorized "from"		e.tsx (Medium)		navigation to /app (from=/app saved). 2. Log in as qa.cliente (CLIENTE role, no /app permission).	ignoring an unauthorized from	on /403 Access Denied instead	-02-redirect-403-tras-login.png)
TC-FE-E2E-02-08 — CLIENTE portal "My Profile" shows personal data	RF-FE-E2E-02	frontend / PerfilPage.tsx (High)	Scripted manual E2E (browser)	1. Log in as qa.cliente. 2. Open "Mi perfil" (/portal).	Name, national ID and email are displayed	Failed — all three fields render blank (API-contract gap)	DEF-03 (Tests/evidence/DEF-03-mi-perfil-vacio.png)
TC-FE-E2E-03-09 — Cancel an ACTIVE ticket with a reason via the UI	RF-FE-E2E-03	frontend / TicketsPage.tsx + tickets (High)	Scripted manual E2E (browser)	1. As RECAUDADOR, open "Anular" on an active ticket. 2. Type a cancellation reason containing a space.	Ticket state becomes ANULADO; valorRecaudado=0	Failed (critical) — modal loses focus on every keystroke and self-closes on a space; ticket is never cancelled from the UI. Backend confirmed correct via direct API call.	DEF-04 (Tests/evidence/DEF-04-modal-anular-foco-perdido.png)

6.8 TRACEABILITY MATRIX (REQUIREMENT ↔ TEST CASE)

Every requirement identified for the in-scope modules is linked to at least one test case, following Metric M-03 (Section 5.1). The table below is the condensed view of the matrix maintained in full detail (including exact test-case identifiers and evidence paths) at Tests/traceability-matrix.csv and Tests/traceability-matrix.xlsx in the project repository.

Requirement ID	Module	Description	Type	Status
----------------	--------	-------------	------	--------

RF-AUTH-01	1. Auth	Login root (happy) 200	Functional API (Postman/Newman)	Executed – Passed
RF-AUTH-02	1. Auth	Login credenciales invalidas 401	Functional API (Postman/Newman)	Executed – Passed
RF-AUTH-03	1. Auth	Login sin password 400	Functional API (Postman/Newman)	Executed – Passed
RF-AUTH-04	1. Auth	GET /me con token 200	Functional API (Postman/Newman)	Executed – Passed
RF-AUTH-05	1. Auth	GET /me sin token 401	Functional API (Postman/Newman)	Executed – Passed
RF-PER-01	2. Personas	Crear persona (happy) 201	Functional API (Postman/Newman)	Executed – Passed
RF-PER-02	2. Personas	Crear persona dni invalido 400	Functional API (Postman/Newman)	Executed – Passed
RF-PER-03	2. Personas	Crear persona sin token 401	Functional API (Postman/Newman)	Executed – Passed
RF-USR-01	3. Usuarios	Register CLIENTE (happy) 201	Functional API (Postman/Newman)	Executed – Passed
RF-USR-02	3. Usuarios	Register username corto 400	Functional API (Postman/Newman)	Executed – Passed
RF-ZON-01	4. Zonas	Crear zona REGULAR (happy) 201	Functional API (Postman/Newman)	Executed – Passed
RF-ZON-02	4. Zonas	Crear zona capacidad 0 -> 400	Functional API (Postman/Newman)	Executed – Passed
RF-ZON-03	4. Zonas	Crear zona con rol CLIENTE -> 403	Functional API (Postman/Newman)	Executed – Passed
RF-ESP-01	5. Espacios	Crear espacio A (AUTO) 201	Functional API (Postman/Newman)	Executed – Passed
RF-ESP-02	5. Espacios	Crear espacio B (AUTO) 201	Functional API (Postman/Newman)	Executed – Passed
RF-ESP-03	5. Espacios	Crear espacio sin idZona 400 (validacion DTO)	Functional API (Postman/Newman)	Executed – Passed
RF-ESP-04	5. Espacios	Crear espacio tipo invalido 400 (enum no permitido)	Functional API (Postman/Newman)	Executed – Passed
RF-ESP-05	5. Espacios	GET espacios disponibles 200	Functional API (Postman/Newman)	Executed – Passed
RF-VEH-01	6. Vehiculos	Crear vehiculo V1 (Auto) 201	Functional API (Postman/Newman)	Executed – Passed
RF-VEH-02	6. Vehiculos	Crear vehiculo V2 (Auto) 201	Functional API (Postman/Newman)	Executed – Passed
RF-VEH-03	6. Vehiculos	Crear vehiculo placa invalida 400	Functional API (Postman/Newman)	Executed – Passed
RF-VEH-04	6. Vehiculos	Crear vehiculo placa duplicada 409	Functional API (Postman/Newman)	Executed – Passed
RF-ASG-01	7. Asignaciones-vehiculos	Asignar V1 -> Cliente 201	Functional API (Postman/Newman)	Executed – Passed

RF-ASG-02	7. Asignaciones-vehiculos	Asignar V2 -> Cliente 201	Functional API (Postman/Newman)	Executed – Passed
RF-ASG-03	7. Asignaciones-vehiculos	Asignar V1 duplicada 409	Functional API (Postman/Newman)	Executed – Passed
RF-ASG-04	7. Asignaciones-vehiculos	GET asignacion por vehiculo V1 200	Functional API (Postman/Newman)	Executed – Passed
RF-TCK-01	8. Tickets (flujo + reglas de negocio)	Ingreso placa1 -> Espacio A 201 (ACTIVO)	Functional API (Postman/Newman)	Executed – Passed
RF-TCK-02	8. Tickets (flujo + reglas de negocio)	[Regla] Vehiculo ya tiene ticket activo 409	Functional API (Postman/Newman)	Executed – Passed
RF-TCK-03	8. Tickets (flujo + reglas de negocio)	[Regla] Espacio A ocupado (2 autos mismo espacio) 409	Functional API (Postman/Newman)	Executed – Passed
RF-TCK-04	8. Tickets (flujo + reglas de negocio)	Ingreso placa inexistente 404	Functional API (Postman/Newman)	Executed – Passed
RF-TCK-05	8. Tickets (flujo + reglas de negocio)	Ingreso sin token 401	Functional API (Postman/Newman)	Executed – Passed
RF-TCK-06	8. Tickets (flujo + reglas de negocio)	Ingreso sin placa 400	Functional API (Postman/Newman)	Executed – Passed
RF-TCK-07	8. Tickets (flujo + reglas de negocio)	GET ticket por id 200	Functional API (Postman/Newman)	Executed – Passed
RF-TCK-08	8. Tickets (flujo + reglas de negocio)	Listar tickets ?estado=ACTIVO 200 (Page)	Functional API (Postman/Newman)	Executed – Passed
RF-TCK-09	8. Tickets (flujo + reglas de negocio)	Pagar ticket 200 (PAGADO, cobro 0.90)	Functional API (Postman/Newman)	Executed – Passed
RF-TCK-10	8. Tickets (flujo + reglas de negocio)	[Regla] Pagar ticket ya pagado 409	Functional API (Postman/Newman)	Executed – Passed
RF-TCK-11	8. Tickets (flujo + reglas de negocio)	[Regla] Anular ticket ya pagado 409	Functional API (Postman/Newman)	Executed – Passed
RF-AUD-01	9. Auditoria (ms-audit)	Crear evento auditoria 201	Functional API (Postman/Newman)	Executed – Passed
RF-AUD-02	9. Auditoria (ms-audit)	Listar auditoria 200	Functional API (Postman/Newman)	Executed – Passed
RF-AUD-03	9. Auditoria (ms-audit)	Auditoria sin token 401	Functional API (Postman/Newman)	Executed – Passed
RF-FE-RBAC-01	Frontend - RBAC	permisosDe: unión de permisos de todos los roles del usuario	Unit (Vitest)	Executed – Passed
RF-FE-RBAC-02	Frontend - RBAC	puede: RECAUDADOR opera tickets pero no gestiona usuarios; CLIENTE/INVITADO sin acceso admin	Unit (Vitest)	Executed – Passed
RF-FE-RBAC-03	Frontend - RBAC	esStaff: distingue roles de panel admin (ROOT/ADMIN/RECAUDADOR) de roles de portal	Unit (Vitest)	Executed – Passed
RF-FE-RBAC-04	Frontend - RBAC	rutaInicial: aterrizaje post-login correcto por rol (dashboard / tickets / portal)	Unit (Vitest)	Executed – Passed
RF-FE-FMT-01	Frontend - Formato	fmtFecha / fmtDinero: manejo de valores nulos/inválidos sin excepción	Unit (Vitest)	Executed – Passed

RF-FE-FMT-02	Frontend - Formato	esCedulaEc: validación cliente de cédula ecuatoriana (espejo de la regla del backend)	Unit (Vitest)	Executed – Passed
RF-FE-FMT-03	Frontend - Formato	rgx: formatos de placa (auto/moto), password, email	Unit (Vitest)	Executed – Passed
RF-FE-GRD-01	Frontend - Guards	RequireAuth: sin sesión redirige a /login; con sesión renderiza la ruta	Unit/Component (Vitest + RTL)	Executed – Passed
RF-FE-GRD-02	Frontend - Guards	RequirePermiso: usuario sin permiso va a /403 (no revela existencia de la ruta); con permiso accede	Unit/Component (Vitest + RTL)	Executed – Passed
RF-FE-GRD-03	Frontend - Guards	HomeRedirect: aterrizaje raíz correcto por rol	Unit/Component (Vitest + RTL)	Executed – Passed
RF-FE-E2E-01	Frontend - C1 Login	Login exitoso y fallido, sesión persistida, logout	Scripted manual functional (E2E)	Executed – Passed
RF-FE-E2E-02	Frontend - RBAC navegación	Navegación admin/portal respeta permisos por rol (ADMIN, RECAUDADOR, CLIENTE)	Scripted manual functional (E2E)	Executed – Passed
RF-FE-E2E-03	Frontend - A10 Operación de Tickets	Flujo completo de emisión y cobro de ticket desde la UI (RECAUDADOR)	Scripted manual functional (E2E)	Executed – Failed
RF-ROL-GAP	Roles (backend)	CRUD de roles (crear/listar/actualizar/activar/desactivar) - existe en Gestion_Parqueaderos pero no en la colección QA	N/A	Not Covered
RF-FE-A2A9-GAP	Frontend - Panel admin (A2-A9)	Pantallas de gestión CRUD (Personas, Usuarios, Roles, Asignación de roles, Zonas, Espacios, Vehículos, Asignaciones) sin caso diseñado todavía	N/A	Not Covered
RF-FE-A11-GAP	Frontend - A11 Auditoría (log)	Pantalla de visualización del log de auditoría	N/A	Not Covered
RF-FE-C2C6-GAP	Frontend - Portal cliente (C2-C6)	Registro, Mi perfil, Mis vehículos, Detalle de vehículo, Disponibilidad (informativo)	N/A	Not Covered

Coverage summary: 53 of 57 identified requirements (93.0%) have at least one executed test case — 52 passed and 1 failed (RF-FE-E2E-03, linked to DEF-04). 4 requirements remain Not Covered (role CRUD via the QA collection, and the frontend admin screens A2–A9/A11 and portal screens C2–C6), consistent with Risk RP-05 (Section 14). This falls short of the ≥95% target set by Exit Criterion X-02 (Section 6.4) and is carried forward as an open item rather than concealed.

SECTION 7. SQA PROBLEM REPORTING AND RESOLUTION

For this academic-scale SQA audit, problem reporting and resolution is implemented as a lightweight defect register rather than the full set of corporate audit forms assumed by the base template. All findings - whether detected during dynamic testing (Section 6) or during static analysis (Section 9) - are logged, classified by severity and priority, and tracked through to verification using the format described in this section. The authoritative, up-to-date defect register is maintained in the project repository at Tests/defects.md, mirrored for this deliverable in Evidencias_Persona4/defects.md, together with screenshot evidence for each finding in the corresponding evidence/ folder.

7.1 PROCESS AUDIT REPORT

Each defect is recorded using a fixed format: an identifier (DEF-XX), the affected module, severity, priority, the test case or manual script where it was detected, reproduction steps, expected vs. actual result, root cause (when identified), a suggested fix, supporting evidence (screenshot or log), and status. Severity is classified as BLOQUEANTE, CRITICA, MAYOR, MENOR, or COSMETICA; priority as ALTA, MEDIA, or BAJA. This replaces the generic Process Audit Report form of the base template, which targets corporate process-compliance audits (e.g., verifying that a requirements-analysis process was followed) rather than the functional/dynamic-testing defects that are the actual output of this audit.

As of this revision, four defects are open and tracked in the register: one cosmetic, one minor, one major, and one critical (see Tests/defects.md, DEF-01 through DEF-04, mirrored as GitHub Issues #10-#13, for full reproduction steps, root-cause analysis, and screenshot evidence). None of the open defects block the primary flows exercised by this audit (login, RBAC-gated navigation, and API-level CRUD operations), although DEF-04 does block the ticket-cancellation flow specifically when performed through the UI.

Table 7-1. Defect Register Summary

ID	Module	Severity	Priority	Detected In	Summary	Status
DEF-01	usuarios (CI-01, backend)	Cosmetic	Low	E2E-01, step 2	Login error message missing accented characters ("contrasena" instead of "contraseña")	Open — GitHub #10
DEF-02	frontend — LoginPage.tsx	Minor (UX)	Medium	E2E-01, step 6b	Post-login redirect follows an unauthorized "from" route, landing the user on /403 instead of a normal session start	Open — GitHub #11
DEF-03	frontend PerfilPage.tsx + usuarios API contract	Major	High	E2E-02, "My Profile"	CLIENTE portal profile screen shows no personal data; no endpoint exposes a	Open — GitHub #12

					client's own name/ID/email	
DEF-04	frontend Modal.tsx + TicketsPage.tsx	Critical	High	E2E-03, step 9	Cancellation modal loses focus on every keystroke and self-closes on a space character, blocking ticket cancellation from the UI	Open — GitHub #13

Bienvenido de nuevo
 Ingresa tus credenciales para continuar

⚠ Usuario o contraseña incorrectos

Usuario *

qa.admin

Contraseña *

.....

Figure 7-4. DEF-01 — Login error message missing accented characters

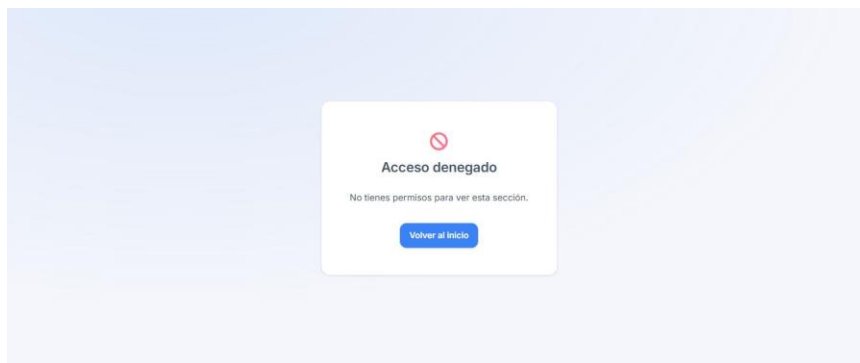


Figure 7-5. DEF-02 — Unauthorized "from" redirect lands on Access Denied after login

Datos personales

Nombre

Cédula

Correo

Teléfono

Nacionalidad

Para cambiar tus datos personales, contacta a la administración.

Figure 7-6. DEF-03 — "My Profile" renders with blank personal-data fields

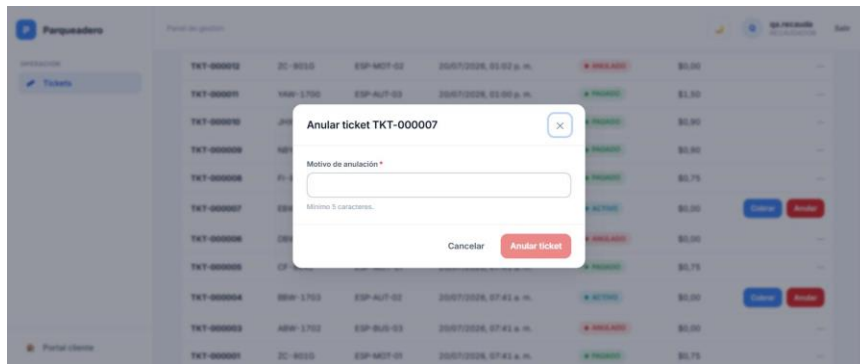


Figure 7-7. DEF-04 — Ticket-cancellation modal loses focus and self-closes

7.1.1 Submittal and Disposition of Process Audit Report

Given the size of the team (four members) and the academic scope of this audit, defect submittal and disposition follow a simplified, single-tier process rather than the multi-level corporate review chain (Senior Management, SEPG, Project Manager) assumed by the base template:

- a. 1. Detection - a defect is found while executing a test case (API via Postman/Newman, unit test via Vitest, or a manual/E2E script) and is logged immediately in Tests/defects.md with severity, priority, reproduction steps, and evidence.
- b. 2. Report - the defect entry is reviewed by the team (informally, given the team size) to confirm severity/priority and to assign it to whichever member owns the affected module for correction.
- c. 3. Correction - the assigned member fixes the code without altering unrelated behavior, following the same non-negotiable constraint used for static-analysis refactors (Section 9).
1. 4. Verification - the original test case (or the manual script that found the defect) is re-executed by the person who reported it; only after it passes is the defect's status updated to CERRADO in the register.
2. Disagreements about severity, priority, or whether a finding is actually a defect are resolved by team discussion; if no agreement is reached, the course instructor (Program Manager, Section 2.1) acts as the deciding authority.

7.1.2 Escalation Procedure for Resolution of Non-Concurrence on Process Audit Report

This escalation path is the same one described in Section 7.1.1 above: unresolved disagreement is escalated to the course instructor, the only management tier above the SQA team for this academic

engagement (see the organization chart in Section 2.1). There is no separate SEPG or multi-project process database to update, since this audit covers a single system and a single course offering.

7.2 RECORDING PROBLEMS IN SOFTWARE CODE OR DOCUMENTATION

Problems found in the audited system's code or documentation are recorded in the same defect register described in Section 7.1, without a separate P/CR or STR form - the fields already captured (module, severity, priority, reproduction steps, root cause, evidence, status) serve the same purpose. Two categories of findings feed this register: (1) issues found during static analysis and configuration review (Section 9), such as RP-02 in the Risk Register (Section 14, Table 14-1) - an unquoted RABBITMQ_ROUTING_KEY value that truncates the audit queue binding because dotenv treats '#' as a comment; and (2) issues found during dynamic/functional testing (Section 6), such as DEF-01 through DEF-04 described above. Each defect in Tests/defects.md is also linked back to the requirement or test case that exposed it in the traceability matrix (Tests/traceability-matrix.xlsx), so every recorded problem can be traced to the specific coverage that found it.

7.3 SOFTWARE TOOL EVALUATION REPORT

Figure 7-2 provides the format for evaluating software tools as described in Section 3.2.

7.4 FACILITIES EVALUATION REPORT

Figure 7-3 provides the format for evaluating existing and planned Parking System — Distributed Microservices facilities as described in Section 3.3.

SOFTWARE TOOL EVALUATION

SQA: Test Strategy and Tooling (P2 — Eduardo Garcia)
2026-07-18

DATE OF EVALUATION:

Software Tool Evaluated:

SonarQube Community Edition — primary static-analysis tool for the Java and TypeScript codebases. Representative entry; the same acceptance process defined in Section 3.2 was applied to every tool listed in Section 8 (ESLint, JUnit 5 + Mockito, JaCoCo, Jest + ts-jest, Vitest + RTL, Postman/Newman, Docker Compose, Git/GitHub).

Methods or criteria used in the evaluation:

Weighted acceptance criteria from Section 3.2: technical compatibility (25%), coverage of the need (25%), reproducibility (20%), traceability/evidence (15%), cost and adoption (10%), integration (5%). Minimum acceptance score: 75/100, with no failure allowed on the compatibility criterion.

Evaluation Results:

Score: 92/100. Runs against the audited commit for both the Java (Maven) and TypeScript (npm) modules; produces a shared, exportable dashboard of bugs, vulnerabilities, code smells, duplication and debt, dated per commit; free Community Edition with low setup overhead; integrates with the existing Git/Maven/npm workflow. No compatibility failure. Accepted for the static-analysis baseline (Section 9).

Recommended Corrective Actions

None blocking. Minor note: SonarQube Community does not natively diff two scans; mitigate by tagging and exporting the “before” and “after” scans separately.

Corrective Action Taken

Separate before/after export procedure adopted and documented for the static-analysis campaign (Section 9). No further action required.



Figure 7-2. Software Tool Evaluation

PROJECT FACILITIES EVALUATION

SQA: SQA Leadership (P1 — Mateo Criollo)

DATE OF EVALUATION: 2026-07-18

Facility Evaluated (Equipment, User/Test/Library Space):

Local development workstations used by the four Team 5 members for this audit, including Docker Desktop hosting the PostgreSQL 16, RabbitMQ, and Kong 3.7 containers defined in docker-compose.yml and gateway/kong.yml (references (h) and (i)).

Methods or criteria used in the evaluation:

Checklist verification against the resource requirements of Section 2.2.1: JDK 25 and Node.js 20+ installed, Docker Desktop running with the required containers healthy, a current version of Google Chrome available to exercise the React frontend, and working access to Git/GitHub, Postman, and SonarQube/SonarCloud.

Evaluation Results:

All four workstations met the checklist before execution began; Docker Desktop availability (Risk RQ-06, Section 14) was verified at the start of each group work session. No blocking gaps found.

Recommended Corrective Actions

None blocking. Because a single shared PostgreSQL instance backs all 6 microservices (Risk RP-03, Section 14), coordinate execution schedules among members to avoid test-data collisions (Risk RQ-04).

Corrective Action Taken

Team adopted a shared execution schedule and the practice of running docker compose down -v between sessions when data isolation was required (Risk RQ-04, Section 14).



Figure 7-3. Project Facilities Evaluation

SECTION 8. TOOLS, TECHNIQUES, AND METHODOLOGIES

The toolchain is selected for compatibility with this audit's mixed-language repository and for its ability to produce repeatable evidence. A single all-in-one tool is avoided where it would hide language-specific detail: SonarQube centralizes the view, while JUnit/Jest/Vitest and JaCoCo/Jest/Vitest coverage keep per-ecosystem precision.

Need	Tool	Scope	Technical Justification	Evidence
Static analysis	SonarQube Community	Java + TypeScript	A shared dashboard for bugs, vulnerabilities, code smells, duplication, and debt per module; enables before/after comparison	Capture/export per commit
TypeScript lint	ESLint	vehiculos, ms-audit, frontend	Native ecosystem rules and a fast complement to SonarQube; run without --fix so the baseline is not altered	Saved output
Java unit testing	JUnit 5 + Mockito	4 Spring Boot services	Already provided by spring-boot-starter-test; suited to services, validators, and isolated rules	Surefire XML/TXT
Java coverage	JaCoCo	4 Spring Boot services	De-facto Maven standard; generates HTML/XML per module. Declared directly in each POM, report goal bound to the verify phase. Result: an initial measurement (2026-07-21) found business-logic coverage far below target on 3 of 4 services; the team closed that gap the same day by porting a dedicated unit-test suite (Appendix C.8) — all 4 services now measure above the X-05 target.	jacoco.xml + HTML
TypeScript unit testing	Jest + ts-jest	2 NestJS services	Already configured in package.json and compatible with unit tests, mocks, and coverage	Jest console + coverage

Frontend unit testing	Vitest + React Testing Library	frontend (CI-07)	Vite's native test runner; covers RBAC guards, route protection, and formatting/validation utilities without a browser	Vitest console + coverage
TypeScript/API integration	Supertest/Jest	NestJS	Validates controllers and HTTP contracts without a browser; dependencies already declared	Jest report
Functional/API	Postman + Newman	System	Reuses versioned collections; supports variables, assertions, and reproducible CLI execution	CLI/HTML/JSON report
System/E2E	Scripted manual testing (frontend); Playwright reserved for future automation	Frontend/Kong	A complex, visible flow complements the API tests; the frontend acts as the client under test, not as code that is unit-audited. Executed as three guided manual scripts (login, RBAC-gated navigation, ticket issuance/cancellation)	Video + screenshots
Environment	Docker Compose	Infrastructure	Reproduces PostgreSQL, RabbitMQ, and Kong with versioned configuration	compose ps + logs
Control	Git/GitHub	Everything	Versions the SUT, tests, reports, and small changes on the QA branch	Hash/PR
Defects	GitHub Issues + Git-versioned Markdown register	Teams	Lightweight for a four-person team; no external account or sync required, lives on the same platform as the code. The Markdown register (Tests/defects.md) holds full detail (root cause, steps, evidence); GitHub Issues #10-#13 mirror	Tests/defects.md (Git history) + github.com/.../issues/10-13 + evidence/*.png

			each entry 1:1 for live state tracking (open/closed), severity/priority labels, and auto-close on the fixing commit. Jira, Trello, and TestLink were equally valid per the course guide's free tool choice; GitHub Issues was chosen because it required no new account or board.	
--	--	--	---	--

Base Reproducible Commands

Objective	Command (from the module directory)
Java tests	.\mvnw.cmd test (Windows) / ./mvnw test (Linux/macOS)
Java coverage	cd <service> && .\mvnw.cmd verify -DskipITs (Windows) / ./mvnw verify -DskipITs (Linux/macOS) - JaCoCo's report goal is bound to the verify phase, not test; report at target/site/jacoco/index.html
NestJS build	npm run build
NestJS tests	npm test -- --runInBand
NestJS coverage	npm run test:cov -- --runInBand
Frontend unit tests	cd frontend && npm run test (single CI run) / npm run test:watch (development)
Frontend coverage	cd frontend && npm run test:cov (vitest run --coverage; requires @vitest/coverage-v8)
Frontend coverage	cd frontend && npm run test:cov (vitest run --coverage; requires @vitest/coverage-v8)
Frontend coverage	cd frontend && npm run test:cov (vitest run --coverage; requires @vitest/coverage-v8)
Frontend manual E2E	cd frontend && npm run dev (port 5173), then follow the guided scripts in Tests/manual/
Auditable lint	npx eslint "{src,apps,libs,test}/**/*.ts" --max-warnings=0 (no --fix)
API via CLI	newman run postman/QA_Parquaderos.postman_collection.json -e postman/Parquadero-Kong.postman_environment.json
Infrastructure	docker compose up -d; docker compose ps

Update (2026-07-21): JaCoCo (0.8.13, required for Java 25 bytecode support) is now declared directly in the four Spring Boot POMs, and @vitest/coverage-v8 was added to the frontend so test:cov runs with real coverage. Both were executed for this revision; results are reported in Section 8's tools table and

Appendix C.8. The frontend manual-E2E and unit-test commands above reflect what P4 actually executed for this audit (Tests/README.md, Evidencias_Persona4).

Update (2026-07-21): SonarCloud analysis was executed across all 7 modules to obtain the "before" and "after" states required by this audit. Result: the baseline capture found 205 issues (3 bugs, 7 vulnerabilities, 195 code smells) and 1,160 minutes of technical debt; following the refactoring commits in Table 9-1, the post-refactor capture found 30 issues (2 bugs, 0 vulnerabilities, 28 code smells) and 187 minutes of technical debt, with all 7 modules passing the Quality Gate. Full per-module tables, dashboard evidence, and residual-finding justification are documented in Appendix D.

Design Techniques

Technique	Application
Equivalence partitioning	Valid/invalid/missing JWT; roles; vehicle types; states
Boundary values	Fares, duration, lengths, capacities, plate formats, and required fields
Decision table	Role × endpoint; vehicle type × space type; state × transition
State transition	Ticket active/closed/voided; assignment active/inactive
Negative testing	Duplicates, non-existent references, malformed data, unavailable service
Pairwise testing	Representative combinations of role, endpoint, and HTTP method when the full space is large

SECTION 9. CODE CONTROL

The purpose of this section is to define the methods and facilities used to maintain, store, secure, and document controlled versions of the identified software during all phases of the software life cycle, whose appropriate use is verified by SQA. For this audit, code control is implemented with Git/GitHub as described below and in Section 1.5.

Code control includes the items listed below:

- a. Identifying, labeling, and cataloging the software to be controlled
- b. Identifying the physical location of the software under control
- c. Identifying the location, maintenance, and use of backup copies
- d. Distributing copies of the code
- e. Identifying the documentation that is affected by a change
- f. Establishing a new version
- g. Regulating user access to the code.

The Parking System — Distributed Microservices uses Git, hosted on GitHub, as its code-control (software configuration management) tool. The project does not maintain a separate, formal Software Configuration Management Plan; the practices described in this section constitute the code-control method verified by SQA for this audit, consistent with Section 1.5.

The seven Configuration Items are organized as a single repository (monorepo) with one top-level directory per CI (usuarios/, zonas/, vehiculos/, asignaciones/, tickets/, ms-audit/, frontend/), which satisfies the identification, labeling, and cataloging of the controlled software. The authoritative copy and its backup reside in the GitHub remote; each team member holds a full-history local clone (the CI checkout uses fetch-depth 0), which serves as both a distributed copy and a recoverable backup. User access to the code is regulated through GitHub repository permissions and a protected branch, with build secrets (SONAR_TOKEN, SONAR_ORGANIZATION) confined to the protected environment of SonarQube Actions.

New versions were established through small, direct commits on the quality branch, proyecto_aseguramiento. Given the four-person scale of this academic engagement, the team worked with single-author commits rather than formal pull requests; each commit message documents the specific SonarCloud rule or coverage gap addressed, and the diff was reviewed by the SQA lead before the next static-analysis run. Table 9-1 lists the representative commits that took the SUT from the "before" state (Section 5.1, metric M-09) to the "after" state, grouped by the category of fix.

Table 9-1. Representative Refactoring and Coverage Commits

Categoría	Hash	Descripción / Commit
Security vulnerabilities	3c88a97	fix(security): eliminar credenciales hardcodeadas - paso 1 vulnerabilidades
Code smells / maintainability	95e7279	fix(sonar): corregir code smells S8688, S3776, S106 en asignaciones, tickets, usuarios y zonas
Code smells / maintainability	f3ceb8c	Step 2: fix S1192, S6809, S5785, S7158 code smells
Code smells / maintainability	203c827	fix(frontend): fix all TypeScript/React Sonar code smells (step 3)

Code smells / maintainability	4bf2633	fix(frontend): corregir issues SonarCloud fase 3 - semantic HTML, ternarios y más
Code smells / maintainability	109109f	refactor(usuarios): remove redundant throws in AuthServiceImplTest.setUp (S1130)
Code smells / maintainability	7a42a3c	refactor: reduce SonarCloud maintainability smells across all modules
Test coverage	3f2b1c1	Fase 1 y fase 2
Test coverage	e50777b	fase 33
Test coverage	83ed54c	fase 4
Test coverage	1e0f2a1	test: add service unit tests for asignaciones and usuarios to raise coverage
Test coverage	8abbc45	test: add service unit tests for tickets to raise coverage
Test coverage	acc8be7	test: add service unit tests for zonas to raise coverage; fix EspacioServicioImpl self-reference constants
Test coverage	35e0928	test: expand zonas coverage to 90%+
Test coverage	13c153d	test: expand asignaciones coverage to 90%+
Test coverage	f685945	test(tickets): add unit tests for service, controller, audit, utils
Test coverage	7b2f34d	test: expand tickets coverage to 90%+
Test coverage	6d21329	test: expand usuarios coverage to 90%+
Test coverage	44562da	fix(tickets): fix TicketControllerTest Page serialization for CI (JDK21)
Test coverage	62dde7c	test: add unit tests across all modules to raise coverage above 90%
Test coverage	c4c0cd5	fix(tests): repair asignaciones/tickets test build breaks blocking coverage
Test coverage	295b1f3	test: add cross-module tests to raise coverage above 90%
Test coverage	b0288bf	test(ms-audit): recurse nested imports to fully cover app.module factories
CI / pipeline	8598574	ci: split sonar step and add retry loop to handle SonarCloud transient errors
CI / pipeline	4b964fb	ci: enable coverage reporting for frontend SonarCloud analysis

SQA (P3) verifies that code control is effective through the continuous-integration pipeline described in Section 8: every push or pull request to the protected branch triggers SonarCloud analysis per stack, and the requirement that the SUT keep compiling was checked at each refactoring step. This automated verification replaces the manual SCM-process audit assumed by the base template (Task 3.19), which is Not Applicable to this academic engagement.

The complete author, timestamp, and diff for each commit above are traceable at https://github.com/JosueXDa/Parquadero_distribuidas/commits?author=MateoJa54 , satisfying the commit-hash evidence required by Table 3-1.

SECTION 10. MEDIA CONTROL

Section 10 (Media Control) is Not Applicable to this audit. The template's media-control items — scheduled backup of physical media, labeled/inventoried storage, and protection from unauthorized access — assume physical computer-program media (tapes, disks, shipped deliverables) that this academic project does not produce: the Parking System is delivered exclusively as source code and configuration versioned in Git/GitHub (Section 9), with no physical media to identify, store, or protect. The equivalent concerns are already covered by GitHub's repository backup and permission model described in Section 9.

SECTION 11. SUPPLIER CONTROL

Section 11 (Supplier Control) is Not Applicable to this audit. The project does not purchase or contract software from external suppliers or vendors. The toolchain used throughout this audit (Section 8) consists exclusively of freely available open-source and academic-tier tools — SonarCloud Community, JaCoCo, Jest, ESLint, GitHub Actions — each evaluated and accepted directly by the SQA team against the criteria of Table 3-2, without a procurement or vendor-support process. No future supplier software is planned for this engagement.

SECTION 12. RECORDS COLLECTION, MAINTENANCE AND RETENTION

Section 12 (Records Collection, Maintenance, and Retention) is Not Applicable in its full, contractor-scale form. This academic audit has no formal Software Development Library (SDL), archival-storage function, or multi-year retention mandate. The SQA records that do exist — the static-analysis reports (Section 4), the defect board, the traceability matrix, and this SQAP itself — are retained instead in the Git repository's version history (Section 9), which persists for the life of the repository at no additional retention cost, well beyond the single academic term for which this plan applies.

SECTION 13. TRAINING

Section 13 (Training) is Not Applicable to this audit in its formal, contractor-scale form. Table 13-1 below is retained from the base template for reference only and does not represent training actually conducted. The four team members already possess the skills required to execute this SQA Plan — Java/Spring Boot, TypeScript/NestJS, React, Git, and SonarCloud — acquired through prior coursework and concurrently with the Software Quality Assurance course (ESPE) itself, rather than through a dedicated classroom/OJT training program. No additional training activities are planned for this engagement.

TABLE 13-1. SQA TRAINING MATRIX (BASE TEMPLATE REFERENCE — NOT EXECUTED)

TASK	SKILL REQUIREMENTS	TYPE	SOURCE
Code Reviews	Source Language, Peer Reviews	Classroom/ OJT	SEPO, Peer Review Process and Workshop
Documentation Reviews	Software Development and Documentation standards and guidelines, Peer Reviews	Classroom/ OJT	SEPO, Peer Review Process and Workshop
Process Audits	Software Development Life Cycle Processes, Audit techniques	Classroom/ OJT	MIL-STD-498, IEEE/EIA 12207
Testing	Testing Methodologies	OJT	
SQA Management	Project Management	Classroom/ OJT	SEPO, Software Project Management (SPM) course
Metrics	Data Collection and Analysis	Classroom/ OJT	SEPO, SPM course
Problem reporting and correction action	Configuration Management	Classroom/ OJT	SEPO, CM Practitioner's Training
Tools	Vendor supplied training	Classroom/ OJT	Vendor
Code, Media, and Supplier Control	Configuration Management	Classroom/ OJT	SEPO, CM Practitioner's Training
Risk Management and Analysis	Risk Management Process	Classroom/ OJT	SEPO, SPM course
Software Management	Software Management Process	Classroom/ OJT	SEPO, Software Management for Everyone (SME) Training, SPM course

SECTION 14. RISK MANAGEMENT

Team 5 identifies, assesses, monitors, and controls risks throughout the audit using an agile, academic-scale risk management process. Risks are identified during the initial technical assessment of the system under test (Section 1.1) and through ongoing findings from static analysis and dynamic testing activities. Each risk is assessed based on its likelihood and impact, assigned a mitigation strategy and a responsible party, and recorded in the Risk Register below. The register is reviewed at each team meeting and updated as new risks are identified or existing ones are closed; changes are recorded in the Change Log at the beginning of this document.

Risks are classified into two categories: product risks, related to possible defects or weaknesses in the audited system itself, and project risks, related to the execution of this software quality assurance (SQA) audit.

TABLE 14-1. PRODUCT RISKS

ID	RISK	PROBABILITY	IMPACT	MITIGATION
RP-01	tickets and ms-audit have no existing test coverage, leaving undetected regressions likely	HIGH	HIGH	Prioritize unit test coverage for these two modules
RP-02	Documented defect: an unquoted RABBITMQ_ROUTING_KEY truncates the audit queue binding (dotenv treats # as a comment)	MEDIUM	MEDIUM	Verify each service. env before starting the environment; add an integration test for the audit event flow
RP-03	A single shared PostgreSQL instance serves all 6 microservices, creating coupling and a single point of failure	LOW	HIGH	Document as a finding in the static analysis report; architectural refactoring is out of scope for this audit
RP-04	JWT keys (keys/jwt_public.pem) are version-controlled in the repository	MEDIUM	MEDIUM	Report as a security code smell in the "before" static analysis report
RP-05	Frontend (React/TypeScript) was added to the SUT after the initial scope definition and has no automated test coverage (no test runner declared in package.json), increasing the risk of undetected regressions in the user-facing layer	HIGH	MEDIUM	Prioritize functional/E2E smoke tests for the frontend's critical flows (login, RBAC-gated admin/portal navigation, ticket issuance) during dynamic testing

TABLE 14-2. PROJECT RISKS

ID	RISK	PROBABILITY	IMPACT	MITIGATION
RQ-01	Limited time available during Parcial 3 to cover 6 microservices plus the newly added frontend	HIGH	HIGH	Scope narrowed and prioritized (Section 1.1); effort focused on tickets/ms-audit
RQ-02	SUT (including the frontend) fails to compile at the start of the audit	LOW	HIGH	Successful compilation of all 6 services was verified before starting any audit work; the frontend build (tsc --noEmit && vite build) was also verified to complete without errors
RQ-03	Lack of consistent test data across environments	MEDIUM	MEDIUM	Reuse the existing scripts/seed_datos.py before executing test runs
RQ-04	Shared environment (single Docker/Postgres instance) among the 4 team members may cause test data collisions	MEDIUM	MEDIUM	Coordinate execution schedules, or run docker compose down -v between sessions
RQ-05	Uneven team familiarity between Java/Spring Boot and TypeScript/NestJS	MEDIUM	LOW	Assign modules to members based on their technical strengths.
RQ-06	Dependency on Docker Desktop being available on each member's machine	LOW	MEDIUM	Verify Docker is running before each group work session
RQ-07	Frontend was added to the SUT after scope, entry/exit criteria, and toolchain (Sections 1.2, 3, 5, 8) were already defined, creating a coherence risk between P1's scope and P2/P3/P4's downstream work	MEDIUM	MEDIUM	Scope Matrix (Table 1-1a) and CI list (Table 1-2) updated to include the frontend before audit execution; Test Strategy and Tooling (P2) must confirm/extend the toolchain in Sections 3, 5, and 8 to cover the frontend

APPENDIX A. LIST OF ACRONYMS

API	Application Programming Interface
CI	Configuration Item
DCR	Document Change Request
DID	Data Item Description
E2E	End-to-End
EIA	Electronic Industries Association
HWCI	Hardware Configuration Item
IDD	Interface Design Description
IEEE	Institute of Electrical and Electronics Engineers
IRS	Interface Requirements Specification
JWT	JSON Web Token
MIL	Military
OJT	On-the-Job
P/CR	Problem/Change Report
RBAC	Role-Based Access Control
SCM	Software Configuration Management
SDD	Software Design Document
SDP	Software Development Plan
SEPO	Systems Engineering Process Office
SME	Software Management for Everyone
SQA	Software Quality Assurance
SRS	Software Requirements Specification
SSS	System/Subsystem Specification
STD	Standard
STR	Software Trouble Report
SUT	System Under Test

APPENDIX B. GENERAL SOFTWARE ENGINEERING PROCESS AUDIT CHECKLISTS

Consistent with the tailoring in Section 3, only the checklists that correspond to in-scope SQA tasks are exercised for this academic audit: Figure B-7 (Software Implementation and Unit Testing, Task 3.10), Figures B-8 and B-9 (Unit Integration, CI/System Qualification Testing, Task 3.11), and Figure B-11 (Corrective Action Process, Task 3.13). The remaining fifteen checklists in this appendix (B-1 through B-6, B-10, and B-12 through B-19) correspond to tasks 3.3-3.9, 3.12, and 3.14-3.21, all marked Not Applicable in Section 3 -- they assume a full requirements/design lifecycle, physical media, subcontractors, and a formal Software Development Library that do not exist for this audit of an already-built system. They are retained below, unmodified, only as the unedited base-template reference (TM-SQA-01); their blank checkboxes are not evidence of anything and must not be read as pending work. Even the four applicable checklists reference concepts -- a Software Development Plan, a Software Development Folder / Unit Development Folder -- that Section 1.5 already notes this project does not have in a formal form; they are kept here for completeness but were not filled in as formal audit records for this academic engagement.

PROJECT PLANNING PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: ☐ Project Plans and commitments exist and are documented in the SDP. ☐ Standards governing the project’s software development process are documented and reflected in the SDP. ☐ The content of the SDP reflects consistent implementation of organization’s standard software process. ☐ The SDP is under configuration management. ☐ The activities of software estimation are conducted in accordance with Software Estimation Process and results are documented. ☐ The organizational database is used for making estimations. ☐ Software requirements are the basis for software plans, work products, and activities. ☐ Plans for conducting software configuration management exists and are documented in the SDP or a separate Software Configuration Management Plan (SCM Plan). ☐ The SCM Plan is under configuration management. ☐ Plans for conducting software quality assurance exists and are documented in the SDP or a separate Software Quality Assurance Plan (SQA Plan). ☐ Plans for conducting software integration testing exists and are documented in a Software Test Plan (STP). ☐ Plans for conducting system testing exist and are documented in a STP ☐ The STP is under configuration management. ☐ Plans for conducting software transition exist and are documented in a Software Transition Plan or Software Development Plan (SDP). ☐ Project schedules and plans undergo a peer review prior to establishing their baselines.

Figure B-1. Project Planning Process Audit Checklist

PROJECT TRACKING AND OVERSIGHT PROCESS AUDIT CHECKLIST
Project:
Date:
Prepared by:
Procedures: ____Project measurements are collected in accordance with the Software Measurement Plan. ____Measurements are used for re-planning analysis. ____Project Lead reviews project status on a biweekly basis. ____Branch Head reviews project status on a monthly basis. ____Division Head reviews project status on a quarterly basis. ____Quarterly Reviews are conducted in accordance with the Software Measurement Plan.

Figure B-2. Project Tracking and Oversight Process Audit Checklist

SYSTEM REQUIREMENTS ANALYSIS PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: ☐ The correct participants are involved in the systems requirements analysis process to identify all user needs. ☐ Requirements are reviewed to determine if they are feasible to implement, clearly stated, and consistent. ☐ Changes to allocated requirements, work products, and activities are identified, reviewed, and tracked to closure. ☐ Project personnel involved in the system requirements analysis process are trained in the necessary procedures and standards applicable to their area of responsibility to do the job correctly. ☐ The commitments resulting from allocated requirements are negotiated and agreed upon by the affected groups. ☐ The commitments are documented, reviewed, accepted, approved and communicated. ☐ Allocated requirements identified as having potential problems are reviewed with the group responsible for analyzing system requirements and documents, and necessary changes are made. ☐ The prescribed processes for defining, documenting, and allocating requirements are followed and documented. ☐ A CM process is in place to control and manage the baseline. ☐ Requirements are documented, managed, controlled, and traced (preferably via a matrix). ☐ The agreed upon requirements are addressed in the SDP.

Figure B-3. System Requirements Analysis Process Audit Checklist

SYSTEM DESIGN PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: ☐ System design documents and the traceability matrix are prepared and kept current and consistent. ☐ Relevant system design documents are updated based on approved requirements changes. ☐ Design walkthroughs (peer reviews) evaluate compliance of the design to the requirements, identify defects in the design, and alternatives are evaluated and reported. ☐ SQA attends a sample set of design walkthroughs. All walkthroughs are conducted. ☐ Defects are identified and resolved. Change control integrity is maintained. ☐ The content of system design documents is selectively reviewed and audited. ☐ Lack of compliance with standards is identified and corrective actions are determined. ☐ Requirements and accompanying design and tools conform to standards, and needed waivers are obtained prior to continuing software development. ☐ Demonstration prototypes comply with requirements and standards. ☐ The demonstration conforms to standards and procedures. ☐ The status of design milestones is reviewed.

Figure B-4. System Design Process Audit Checklist

SOFTWARE REQUIREMENTS ANALYSIS PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: Part 1. Software Requirements ☐ Software requirements are documented in a Software Requirements Specification (SRS) or other approved format and are included in a traceability matrix. ☐ The SRS is maintained under configuration management. ☐ The SRS changes undergo a peer review before they are incorporated into the requirements baseline. ☐ Ensure that the peer review validates testability of requirements and that appropriate metrics are established to validate measurable performance requirements. ☐ Software development plans, work products, and activities are changed to be consistent with changes to the software requirements. ☐ Software requirements analysis techniques are consistent with the SDP. ☐ Automated tools acquired to manage software requirements trouble reports and change requests are correctly used. ☐ Software engineering group is trained to perform requirements management activities. ☐ Measurements are made and used to determine the status of requirements management. Part 2. Interface Requirements ☐ Interface requirements are documented in an Interface Requirements Specification (IRS) or other approved format. ☐ The IRS is maintained under configuration management. ☐ The IRS changes undergo peer review before they are incorporated into the requirements baseline. ☐ Software development plans, work products, and activities are changed to be consistent with changes to the interface requirements. ☐ Automated tools are used to manage interface requirements trouble reports and change requests. ☐ Software engineering group is trained to perform requirements management activities.

Figure B-5. Software Requirements Analysis Process Audit Checklist

SOFTWARE DESIGN PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: Part 1. Software Design ____The following documents undergo peer review during this phase of development: ____Software Design Document ____Interface Design Document ____Software Test Plan (Test Ids, Test Cases) ____Software Programmers Manual ____Software Test Description ____Firmware Support Manual ____The following modified documents are placed under CM during this phase of development: ____Software Design Document ____Interface Design Document ____Software Test Plan (Test Ids, Test Cases) ____Software Programmers Manual ____Software Test Description ____Firmware Support Manual ____Design documents and the traceability matrix are prepared and kept current and consistent based on approved software requirement changes. ____Design walkthroughs evaluate compliance of the design to the requirements, identify defects in the design, and alternatives are evaluated and reported. ____Design walkthroughs are conducted in accordance with Peer Review Process. ____Changes to the software design are identified, reviewed, and tracked to closure. ____Software design is consistent with the design methodology approved in the SDP.

Figure B-6. Software Design Process Audit Checklist

SOFTWARE DESIGN PROCESS AUDIT CHECKLIST (cont)
Project: Date: Prepared by:
Procedures: Part 1. Software Design (cont) ☐ The method, such as the Software Development Folder or Unit Development Folder, used for tracking and documenting the development/maintenance of a software unit is implemented and is kept current. Part 2. Interface Design ☐ Interface Requirements are documented in an Interface Design Document (IDD) or other approved format. ☐ The IDD is maintained under configuration management. ☐ The IDD changes undergo peer review.

Figure B-6. Software Design Process Audit Checklist (continued)

SOFTWARE IMPLEMENTATION AND UNIT TESTING PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: Part 1. Software Implementation ☐ Code and the traceability matrix are prepared and kept current and consistent based on approved software requirement changes. ☐ Code walkthroughs (peer review) evaluate compliance of the code to the approved design, identify defects in the code, and alternatives are evaluated and reported. ☐ Code walkthroughs are conducted in accordance with Peer Review Process. ☐ Changes to code are identified, reviewed, and tracked to closure. ☐ Code is maintained under configuration management. ☐ Code changes undergo peer review before they are incorporated into the software baseline. ☐ Software coding is consistent with the coding methodology approved in the Software Development Plan (SDP). ☐ The method, such as the Software Development Folder or Unit Development Folder, used for tracking and documenting the development/maintenance of a software unit is implemented and is kept current. Part 2. Unit Testing ☐ Software unit testing is conducted in conformance with the approved standards and procedures described in the SDP. ☐ Ensure that passing criteria for unit test is documented, and that compliance has been recorded. ☐ Results of unit testing are documented in the Software Development Folder or Unit Development Folder.

Figure B-7. Software Implementation and Unit Testing Process Audit Checklist

UNIT INTEGRATION AND TESTING PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: Part 1. Configuration Identification ☐ The CIs are integrated under change control. If not, go to Part 2; otherwise, continue. ☐ The CIs integrated into the system were obtained from an authorized Configuration Management representative in accordance with the SDP. ☐ The baseline versions of each CI are integrated into the system. ☐ All CI components of software integration are under change control in accordance with the SCM Plan. ☐ If yes, describe how they are identified. Part 2. Integration Process ☐ A plan for the integration of the CIs exists. ☐ The plan specifies the order and schedule in which the CIs are integrated. ☐ The CIs are integrated in accordance with the schedule and in the specified order. ☐ The integration plan specifies which version of each CSC and CSU is to be integrated. ☐ The correct version is integrated. ☐ The integrated CIs have completed unit testing. ☐ Any required corrections have been completed. ☐ The CIs have been retested. ☐ Test procedures are defined for CI integration. ☐ The defined procedures are followed.

Figure B-8. Unit Integration and Testing Process Audit Checklist

UNIT INTEGRATION AND TESTING PROCESS AUDIT CHECKLIST (cont)
Project: Date: Prepared by:
Procedures: Part 2. Integration Process (cont) ____ Test cases are defined. ____ The defined test cases are followed. ____ Test pass/fail criteria are defined. ____ The defined test pass/fail criteria are followed. ____ The test results are documented in Unit Development Folders.

Figure B-8. Unit Integration and Testing Process Audit Checklist (continued)

CI INTEGRATION TESTING, AND SYSTEM QUALIFICATION PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: Part 1. Test Plan ☐ An approved test plan and test descriptions exists. ☐ The plan and descriptions used are under configuration management control. ☐ The latest version of the plan and description are used. Part 2. Testing Process ☐ The system software was received from an authorized configuration management source. ☐ The test environment, including both hardware and software requirements, is set up as required by the test plan. ☐ The tests were performed in the correct order. ☐ Each test case in the test description was executed. ☐ The system is tested after each CI is integrated. ☐ Test passing criteria is documented and compliance is recorded. ☐ The results of the tests are recorded in a test report. ☐ If yes, describe the information that is recorded and where it is recorded. ☐ CIs are retested after integration to assure they still satisfy their requirements without interference from remainder of system. ☐ The system that results from integration of each CI is placed under configuration management control. ☐ If yes, describe how it is identified.

Figure B-9. CI Integration Testing and System Qualification Process Audit Checklist

CI INTEGRATION TESTING, AND SYSTEM QUALIFICATION PROCESS AUDIT CHECKLIST (cont)
Project: Date: Prepared by:
Part 3. Trouble Reporting ___ The discrepancies found are entered into a P/CR or STR Configuration Management system for change control. If not, describe where they are they recorded. ___ The entries are completed at the time the discrepancies are found. ___ The P/CR or STR's reference number is kept in the test file. If not, describe where it is kept. ___ P/CR or STRs are written when problems are found in the test environment, test plan, test descriptions, or test cases. ___ These P/CR or STRs are sent through the same change control process as software discrepancies. Part 4. Modifications ___ Are modifications or corrections made to the test environment during testing? ___ If yes, were the modifications approved through the change control process prior to implementation? ___ What documentation of the modifications exists? ___ What are the changes? ___ Are modifications or corrections made to the test descriptions during testing? ___ If yes, were the modifications approved by the change control process prior to implementation? ___ What documentation of the modifications exists? ___ What is the approving LCCB date? _____ ___ What is the STD change release date? _____ ___ Were the change control procedures in the SCM Plan followed? ___ If no, what are the changes? Part 5. Completion Criteria ___ All specified CIs have been integrated into the system.

Figure B-9. CI Integration Testing and System Qualification Process Audit Checklist. (continued)

CI INTEGRATION TESTING, AND SYSTEM QUALIFICATION PROCESS AUDIT CHECKLIST (cont)
Project: Date: Prepared by:
Part 5. Completion Criteria. (cont) ____All test cases have been executed on the system. ____All P/CRs or STRs are closed out. ____If not, all outstanding P/CRs or STRs are properly documented in the VDD. ____The Software Test Report has been completed and approved. ____The Software Test Report has been placed under change control. ____The appropriate authority has determined whether the system passed or failed integration testing. ____List the individual or group that determined whether the system passed or failed. ____Describe how the pass or fail determination was made. ____The software system is ready to be integrated with operational system. Part 6. Software Development Files ____The Software Test Report includes any retests due to software failures. ____If yes, list the failures with their corresponding P/CR or STR reference numbers. ____Using the P/CR or STR CM system, list all the CIs changed due to these failures. ____All the software development files of the listed CIs were updated in accordance with SDP. ____If not, list all software development files that were not updated.

Figure B-9. CI Integration Testing and System Qualification Process Audit Checklist. (continued)

CI INTEGRATION TESTING, AND SYSTEM QUALIFICATION PROCESS AUDIT CHECKLIST (cont)
Project: Date: Prepared by:
Part 7. Software Test Report Accuracy ☐ The Software Test Report supplies the Configuration Identification Number (CIN) for all test documents (STP, STD) and software. If not, the Software Test Report is incomplete. ☐ The tester can run the evaluation tests with the specified CINs? If not, the Software Test Report is inaccurate. ☐ The results of these tests match the Software Test Report? If not, the Software Test Report is inaccurate.

Figure B-9. CI Integration Testing and System Qualification Process Audit Checklist. (continued)

END-ITEM DELIVERY PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: Part 1. Audits ☐ A functional audit was conducted, if required. ☐ A physical audit was conducted, if required. Part 2. Package Generation ☐ The software is generated from the software library in accordance with the SDP. ☐ If yes, the software is the latest version of the software in the software library. ☐ If not, why not? ☐ The documentation is generated from masters controlled by the Configuration Management personnel as required by the SCM Plan. Part 3. Delivery Package ☐ The software media is labeled correctly, showing at a minimum software name, release date, and correct version number. The checklist in Figure B-12 contains more details on media labeling. ☐ The Version Description Document is with the software media. ☐ If yes, the Version Description Document is the correct version for the software. ☐ The Version Description Document has been formally inspected. ☐ The User's Manual is with the software media. ☐ If yes, the User's Manual is the correct version for the software. ☐ The User's Manual has been formally inspected.

Figure B-10. End-Item Delivery Process Audit Checklist

END-ITEM DELIVERY PROCESS AUDIT CHECKLIST (cont)
--

Project: Date: Prepared by:
Procedures: Part 4. Media Distribution List ____ A distribution list for the deliverable exists. ____ If yes, it is complete, all organizations listed, all addresses correct and current. ____ If yes, are any organizations listed that do not need to receive deliverable? ____ Is the deliverable classified? ____ If yes, the personnel on the distribution list have required clearance and need-to-know. Part 5. Packaging ____ The packaging material is suitable for the contents and transmission method used. ____ Does package contain a signed transmittal letter? ____ If yes, the transmittal information is correct. ____ All contents are listed on transmittal contained in package. ____ Does package include receipt acknowledgment form? Part 6. Problem Notification ____ There is a specified method for the receiving organization to notify the sender of problems and deficiencies in the package. ____ If yes, describe the method. ____ There is a specified method for logging and handling distribution problems. Describe it. ____ Distribution problems are handled by a specific person. Identify that person.

Figure B-10. End-Item Delivery Process Audit Checklist (continued)

SOFTWARE CORRECTIVE ACTION PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: Part 1. Implementation of a closed loop Corrective Action (CA) Process ☐ The CA Process is a closed-loop. ☐ If yes, does the closed-loop CA Process verify the items listed below? ☐ All detected problems are promptly reported. ☐ All detected problems are entered into CA Process. ☐ Action is initiated on problems. ☐ Resolution is achieved. ☐ Status is tracked and reported. ☐ Records are maintained. ☐ Problem/change/discrepancy reports are the input. ☐ If the CA was not resolved with the Project Manager, the problem was escalated to the appropriate management personnel for resolution, and the final disposition was recorded and filed. Part 2. Inputs to the Corrective Action Process ☐ A CA Process exists. Location: ☐ The CA Process is documented. Location: ☐ The CA Process is implemented. Notes:

Figure B-11. Software Corrective Action Process Audit Checklist

SOFTWARE CORRECTIVE ACTION PROCESS AUDIT CHECKLIST (cont)
Project: Date: Prepared by:
Part 3. Classification of Problems by Category and Priority ___ Problems are classified by category. Categories include the following. ___ Software Problem. The software does not operate according to supporting documentation and the documentation is correct. ___ Documentation Problem. The software does not operate according to supporting documentation but the software operation is correct. ___ Design Problem. The software does not operate according to supporting documentation but a design deficiency exists. ___ Problems are classified by priority. Priorities include the following. ___ Priority 1: A software problem that does one of the following: - Prevents the accomplishment of an operational or mission essential capability specified in the baseline requirements. - Prevents the operator's accomplishment of an operational or mission essential capability. - Jeopardizes personnel safety. ___ Priority 2: A software problem that does one of the following: - Adversely affects the accomplishment of an operational or mission essential capability specified in the baseline requirements so as to degrade performance and for which no alternative work-around solution is known. - Adversely affects the operator's accomplishment of an operational or mission essential capability for which no alternative work-around solution is known. ___ Priority 3: A software problem that does one of the following: - Adversely affects the accomplishment of an operational or mission essential capability specified in the baseline requirements so as to degrade performance and for which an alternative work-around solution is known. - Adversely affects the operator's accomplishment of an operational or mission essential capability for which an alternative solution is known.

Figure B-11. Software Corrective Action Process Audit Checklist (continued)

SOFTWARE CORRECTIVE ACTION PROCESS AUDIT CHECKLIST (cont)
Part 3. Classification of Problems by Category and Priority (cont) ____ Priority 4: A software problem that is an operator inconvenience or annoyance and which does not affect a required operational or mission essential capability. ____ Priority 5: All other errors. Part 4. Performance of Trend Analysis. ____ Analysis is performed to determine problem areas. ____ Underlying factors/root causes are identified, categorized, and prioritized. ____ Resources are expended in finding and treating root causes. Part 5. Evaluation of Corrective Action taken ____ Corrective actions are evaluated to verify the items listed below: ____ Problems have been resolved. ____ Adverse trends have been reversed. ____ Changes have been correctly implemented. ____ Introduction of no additional problems. NOTES:

Figure B-11. Software Corrective Action Process Audit Checklist (continued)

MEDIA CERTIFICATION PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: Part 1. Media Production ____Media containing source code and media containing the object code that are delivered correspond to one another. ____A documented process that is used to implement production of media from the software library exists. ____If not, a plan is being created or a reason why one is not needed is being documented. Skip to Part 2. ____The plan was followed in production of media. ____Software was created from correct files in the software library by CM personnel. ____Documents were created from approved master copies by CM personnel. Part 2. Media Labeling ____There is a documented standard that is followed in labeling the media. ____If yes, describe the standard method used to identify the product, version, and Configuration Identification Number. ____Media is clearly labeled. ____The label contains all required information (product, version, and Configuration Identification Number). ____If software is classified, the media clearly reflects the correct classification. ____Software document is clearly labeled with product, CIN, and version number, if applicable.

Figure B-12. Media Certification Process Audit Checklist

MEDIA CERTIFICATION PROCESS AUDIT CHECKLIST (cont)
Project: Date: Prepared by:
Part 3. Media Contents ☐ A listing of contents on the media exits. ☐ If yes, describe where the listing is located. ☐ The media contains the contents specified in the listing. ☐ The contents of the media match the label information, i.e., is it the correct version for the correct hardware platform? ☐ The documents contain all change pages required for this version of the documents.

Figure B-12. Media Certification Process Audit Checklist (continued)

NON-DELIVERABLE SOFTWARE CERTIFICATION PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: ____ Deliverable software is dependent on non-deliverable software. ____ If yes, provision is made so acquirer has or can obtain non-deliverable software. ____ Non-delivered software performs it's intended use. ____ Non-delivered software is placed under configuration management.

Figure B-13. Non-Deliverable Software Certification Process Audit Checklist

STORAGE AND HANDLING PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: ____ Documents and media are stored according to the Software Development Library procedure. ____ Storage areas for paper products are free from adverse environmental effects (high humidity, magnetic forces, heat, and dust) ____ Storage areas for media products are free from adverse environmental effects (high humidity, magnetic forces, heat, and dust) ____ Storage containers for classified material are appropriate for level of classified material.

Figure B-14. Storage and Handling Process Audit Checklist

SUBCONTRACTOR CONTROL PROCESS AUDIT CHECKLIST	
Project:	
Date:	
Prepared by:	
Procedures:	
☐	A subcontract manager is designated to be responsible for establishing and managing the software subcontract.
☐	Subcontract manager is trained to perform these activities.
☐	The work to be subcontracted is defined and planned according to a documented procedure.
☐	The subcontract SOW is reviewed and approved by the project manager, branch head, and division head.
☐	The subcontract SOW is managed and controlled.
☐	The subcontractor is selected according to a documented procedure.
☐	The contractual agreement between the prime contractor and the software subcontractor is used as the basis for managing the subcontract. The contractual agreement documents the items listed below:
☐	The terms and conditions
☐	SOW
☐	Requirements for the products to be developed.
☐	List of dependencies between subcontractor and prime.
☐	Subcontracted products to be delivered to the prime.
☐	Conditions under which revisions to products are to be submitted.
☐	Acceptance procedures and acceptance criteria to be used in evaluating the subcontractor products before they are accepted by the prime.
☐	Procedures and evaluation criteria to be used by the prime to monitor and evaluate the subcontractor's performance.

Figure B-15. Subcontractor Control Process Audit Checklist

SUBCONTRACTOR CONTROL PROCESS AUDIT CHECKLIST (cont)	
Project:	
Date:	
Prepared by:	
Procedures:	____ Subcontractor’s software development plan is reviewed/approved by the prime. ____ Approved subcontractor's SDP is used for tracking the software activities and communicating status. ____ Changes to the subcontractor's SOW are resolved according to a documented procedure. ____ Project manager conducts periodic status/coordination reviews with the subcontractor’s management. ____ Periodic technical reviews and interchanges are held with the subcontractor. ____ Formal reviews to address the subcontractor’s accomplishments and results are conducted at selected milestones. ____ Reviews are documented in the SOW. ____ Reviews address status of subcontractor software activities. ____ Significant issues, action items, and decisions are identified and documented. ____ Software risks are addressed. ____ Subcontractor’s SDP is refined as appropriate. ____ The prime contractor's software quality assurance group monitors the subcontractor’s quality assurance activities. ____ The prime contractor conducts acceptance testing of subcontractor products. ____ Subcontractor’s performance is evaluated on a periodic basis, and reviewed with the subcontractor. ____ Measurements are made and used to determine the status of the subcontract. ____ The activities of the subcontract are reviewed by the Division Head on a quarterly basis.

Figure B-15. Subcontractor Control Process Audit Checklist (continued)

DEVIATION AND WAIVER PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: ____ Deviations and waivers are prepared in accordance with the documented procedures in the project SCM Plan. ____ Deviations and waivers are reviewed and approved by the appropriate personnel in accordance with the project SCM Plan ____ Records of deviations and waivers are maintained and reflect current development configuration.

Figure B-16. Deviations and Waivers Process Audit Checklist

SOFTWARE CONFIGURATION MANAGEMENT PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: Part 1. SCM Plan _____Project follows the organizational policy for implementing SCM. _____Group responsible for coordinating and implementing SCM for the project exists. _____A documented and approved SCM Plan is used as the basis for performing SCM activities. _____Configuration control of changes to baseline documents and software are managed in accordance with the SCM Plan. _____A configuration management library system is established as the repository for the software baseline. _____The CM library is the single place of storage for the baseline version of all software. _____Access to software products in the CM library is in accordance with the Library Control procedures. _____Software work products to be placed under SCM are identified according to the SCM Plan. _____Local Change Control Board (LCCB) exists and implements LCCB procedures. _____Change request and problem reports for all configuration items are handled in accordance with the PCR procedure. _____Changes to baselines are controlled according to the SCM Plan, LCCB procedure, and PCR procedure. _____Products from the software baseline library are created and their release is controlled according to the Library Control procedures. _____Configuration status accounting reports are prepared in accordance with the SCM plan

Figure B-17. Software Configuration Management Process Audit Checklist

SOFTWARE CONFIGURATION MANAGEMENT PROCESS AUDIT CHECKLIST (cont)
Project: Date: Prepared by:
Part 2. Configuration Identification ☐ Product Baselines and the Developmental Library can be identified. ☐ If yes, describe the method used to identify the Baselines and the Developmental Library. ☐ List the documents that make up these Baselines and Developmental Library. ☐ The documentation and the computer storage media containing code, documentation, or both can be identified. ☐ If yes, describe the method used to identify the documentation and the computer storage media and list the documents that are placed under configuration control. ☐ Each CI and its corresponding components can be identified. ☐ If yes, describe the method used to identify them. ☐ A method is used to identify the name, version, release, change status, and any other identification details of each deliverable item. ☐ If yes, describe the method used. ☐ For each customer, identify the deliverable item, version, release, and change status being used. ☐ A method is used to identify the version of each CI to which the corresponding software documentation applies. ☐ If yes, describe the method used. ☐ List the SRS and SDD for each CI. ☐ A method is used to identify the specific version of software contained on a deliverable medium, including all changes incorporated since its previous release. ☐ If yes, describe the method used. ☐ The deliverable medium matches the CM masters? List any discrepancies.

Figure B-17. Software Configuration Management Process Audit Checklist (Continued.)

SOFTWARE CONFIGURATION MANAGEMENT PROCESS AUDIT CHECKLIST (cont)
Project: Date: Prepared by:
Part 3. Configuration Control ___An established plan for performing configuration control exists. ___If yes, there is a method to establish a Developmental Library for each CI. ___If yes, describe the method used. ___List the CIs in the Developmental Library. ___A method exists to maintain current copies of the deliverable documentation and code. ___If yes, describe the method used. ___List the current copies. List all discrepancies. ___A method exists to control the preparation and dissemination of changes to the master copies of deliverable software and documentation. ___If yes, describe the method used. ___Master copies of deliverables reflect only approved changes. List any discrepancies. ___List the changes in current deliverable software/documents. Part 4. Configuration Status Accounting ___A documented plan for implementing and performing configuration status accounting exists. ___There are status reports on all products comprising the Developmental Libraries and the Functional Allocated and Product Baselines. ___Proposed and implemented changes to a CI and its associated configuration identification documents are recorded and reported. ___If yes to two out of three, answer the following. If not, then go to Part 5.

Figure B-17. Software Configuration Management Process Audit Checklist (continued)

SOFTWARE CONFIGURATION MANAGEMENT PROCESS AUDIT CHECKLIST (cont)
Project: Date: Prepared by:
Part 4. Configuration Status Accounting (cont) ____ Describe the method used to provide traceability of changes to controlled products. ____ Describe the method used for communicating the status of configuration identification and associated software. ____ Describe the method for ensuring that delivered documents describe and represent the associated software. Part 5. Engineering Change Proposals ____ Engineering Change Proposals (ECPs) are prepared in accordance with the appropriate standard. ____ Software Change Notices (SCNs) are prepared in accordance with the appropriate standard. ____ Describe the method used for handling requested changes to the CI. ____ Describe the method used to authorize SCNs and ECPs.

Figure B-17. Software Configuration Management Process Audit Checklist (continued)

SOFTWARE DEVELOPMENT LIBRARY CONTROL PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: Part 1. Establishment of SDL Environment and Control ___ The SDL is established and procedures to govern its operation exist. ___ The SDL provides a positive means for recognizing related elements (i.e., those versions that constitute a particular baseline and protecting the software against destruction or unauthorized modification). ___ Documentation and computer program materials approved by the LCCB are placed under library control. ___ All software, tools, and documentation relevant to the software development are placed under library control. ___ Published procedures/standards for the SDL exist. ___ SDL procedures include identification of persons/organization responsible for receiving, storing, controlling, and disseminating library materials. ___ Access to SDL is limited to authorized personnel. ___ If yes, describe the procedures used to limit access. ___ Safeguards are in place to assure that no unauthorized alterations are made to controlled material. ___ If yes, describe those safeguards. ___ Describe or list the materials to be controlled. ___ Describe how the materials are approved and placed under control: ___ Describe how changes to the software part are handled and how changes to lines of code are identified. ___ The SDL contains master copies of each CI under Computer Program Library control.

Figure B-18. Software Development Library Control Process Audit Checklist

SOFTWARE DEVELOPMENT LIBRARY CONTROL PROCESS AUDIT CHECKLIST (cont)
Project: Date: Prepared by:
Part 1. Establishment of SDL Environment and Control (cont) ____ Periodic back up of the software is performed to prevent loss of information due to any failure of the Development Library System. ____ If yes, describe the backup procedure and frequency of backups. Part 2. Assurance of Controlled Material Validity ____ Duplications from controlled and tested master copies are verified before delivery as exact copies. ____ All deliverable software products that are duplicated from controlled and tested master copies are compared with that master copy to assure exact duplication. ____ Describe how identification numbers and revision codes are assigned to controlled documents and software. ____ Describe the way releases of controlled materials are recorded. ____ List the people or organization responsible for assurance of software media validity. ____ A formal release procedure exists. If so, describe it? ____ The material contained in the library is promptly and correctly updated when a change to any of these materials is authorized.

Figure B-18. Software Development Library Control Process Audit Checklist (continued)

NON-DEVELOPMENTAL SOFTWARE PROCESS AUDIT CHECKLIST
Project: Date: Prepared by:
Procedures: ____Non-developmental software performs its intended functions. ____Non-developmental software is placed under internal CM. ____Data rights provisions and licensing is consistent with the SDP.

Figure B-19. Non-Developmental Software Process Audit Checklist

APPENDIX C. TEST SUMMARY REPORT (INFORME DE CIERRE)

This appendix consolidates the closing metrics for the dynamic-testing campaign described in Section 6 and Section 7, in the format required by the course rubric (Block C — Informe de Cierre). It complements the narrative defect analysis of Section 7 with the quantitative view requested by Metrics M-01 through M-08 (Section 5.1). Run date: 2026-07-20. Environment: local Docker (PostgreSQL 16, RabbitMQ, Kong 3.7), the six backend microservices, and the React/TypeScript frontend. Full raw evidence is kept in Tests/test-summary.md, Tests/defects.md and Tests/evidence/ in the project repository.

C.1 EXECUTIVE SUMMARY

Suite	Cases	Passed	Failed	% Pass
Functional API (Postman/Newman)	40	40	0	100%
Frontend unit tests (Vitest + RTL)	36	36	0	100%
E2E-01 Login (manual)	7	6	1	86%
E2E-02 RBAC navigation (manual)	8	7	1	87.5%
E2E-03 Ticket issuance/cancellation (manual)	11	10	1	91%
Total	102	99	3	97.1%

Total assertions in the API suite: 117/117 passed. Defects found: 4 (Section 7, Table 7-1).

C.2 RESULTS BY MODULE (BACKEND API)

Module (CI)	Cases	Passed	Failed
Auth / usuarios (CI-01)	5	5	0
Personas	3	3	0
Usuarios (registration)	2	2	0
Zonas (CI-02)	3	3	0
Espacios	5	5	0
Vehiculos (CI-03)	4	4	0
Asignaciones-vehiculos (CI-04)	4	4	0
Tickets (CI-05)	11	11	0
Auditoria (CI-06)	3	3	0

C.3 RESULTS BY AREA (FRONTEND, CI-07)

Area	Cases	Passed	Failed	Type
RBAC (rbac.ts)	16	16	0	Unit (Vitest)

Format/validation (format.ts)	18	18	0	Unit (Vitest)
Route guards (guards.tsx)	5	5	0	Unit/Component (Vitest + RTL)
Login / session (E2E-01)	7	6	1	Manual (browser)
RBAC navigation in UI (E2E-02)	8	7	1	Manual (browser)
Ticket issuance/cancellation in UI (E2E-03)	11	10	1	Manual (browser)

C.4 DEFECT DENSITY

Module	Defects	Cases Executed	Density
Backend API (6 microservices)	0	40	0.00
Frontend — unit tests (RBAC/format/guards)	0	36	0.00
Frontend — manual E2E (login, RBAC nav, tickets)	4	26	0.15 defects/case
Total	4	102	0.039 defects/case

Two defects are low/medium severity (DEF-01, DEF-02); two are major/critical severity with high priority: DEF-03 (My Profile shows no personal data due to an API-contract gap) and DEF-04 (the ticket-cancellation modal is unusable in practice, blocking a core business operation from the UI). Neither blocks login or read-only browsing, but DEF-04 does block cancelling a ticket from the interface.

C.5 COVERAGE AND KNOWN GAPS

No formal QA coverage: Role CRUD (present in the development collection `Gestion_Parqueaderos`, not replicated in the QA collection). No coverage beyond unit level for the frontend admin screens A2–A9/A11 and portal screens C2–C6 (see docs/frontend-pantallas.md). The three manual scripts in `Tests/manual/` cover the highest-risk flows (login, RBAC, tickets) but not the remaining screens. This is consistent with Risk RP-05 (Section 14): the frontend was added to scope late, with partial coverage. As a result, requirement coverage (Metric M-03) stands at $53/57 = 93\%$, short of the $\geq 95\%$ target set by Exit Criterion X-02 (Section 6.4); this gap is carried forward as an open item rather than concealed.

C.6 CONCLUSION OF THIS RUN

The backend (6 microservices), audited through the existing QA collection, presents no functional defects detectable with the 40 designed cases; direct API calls further confirmed that the tickets business operations (charge, cancel) work correctly at that level. The frontend’s critical security logic (role-based access control) is covered and green at the unit level, and manual browser verification (E2E-01, E2E-02) confirms that RBAC enforcement also holds in the real UI — no lower-privilege role was able to reach a restricted section.

Four real defects were found during manual execution. The two most relevant to prioritize before delivery: DEF-04 (critical) — the ticket-cancellation modal is unusable in practice, the most significant finding of this round because it directly affects tickets (CI-05), the module the team had already flagged

as highest risk (RP-01) before this audit; and DEF-03 (major) — the client portal's "My Profile" screen shows no personal data due to a mismatch between what the usuarios API exposes and what the frontend expects. The other two (DEF-01 cosmetic, DEF-02 UX) are low impact. With the three manual executions (E2E-01, E2E-02, E2E-03) complete, the remaining admin/portal screens (Section 6.8 traceability matrix, rows marked Not Covered) are the only item pending before declaring Ejecución y Evidencias fully closed for CI-07.

C.7 CLOSURE VERDICT AGAINST EXIT CRITERIA (SECTION 6.4)

This section renders the explicit closure decision that Section 6.4 requires, evaluated against the results reported in C.1–C.4 and the defect register (Section 7, Table 7-1). It is grounded in the current state of the audited repository, verified on 2026-07-21: the DEF-04 defect is still reproducible in frontend/src/ui/Modal.tsx and frontend/src/pages/admin/TicketsPage.tsx (the useEffect focus-management dependency and the unmemoized onClose prop described in Table 7-1 are both still present in the current commit), and all 4 services and the frontend now measure above the X-05 coverage target (Appendix C.8).

Criterion	Result	Basis
X-01 — Execution (M-01)	Met	All 3 manual E2E scripts run to completion; all critical modules (tickets, usuarios, RBAC) have executed cases (Sections 6.7–6.8).
X-02 — Requirement coverage (M-03)	Partially met	100% of requirements flagged High priority in Section 5.1.1 have an executed case; overall coverage is $53/57 = 93\%$, short of the $\geq 95\%$ overall bar (Section 6.8).
X-03 — Pass rate (M-02)	Not met on critical cases	Overall pass rate is 97.1% ($\geq 90\%$), but the failing case TC-FE-E2E-03-09 (cancel a ticket with a reason) is on tickets, a High-priority module — the "100% of critical cases passed" sub-clause is not satisfied, and the exception in X-03 applies only to non-critical cases.
X-04 — Open Blocker/Critical defects (M-07)	Not met	DEF-04 is open and classified Critical (Table 7-1); it blocks ticket cancellation from the UI, a core business operation on the highest-risk module (RP-01, Section 14).
X-05 — Line/branch coverage (M-04/M-05)	Met	Re-measured 2026-07-21 with JaCoCo 0.8.15 and Vitest v8, after the team closed the gap first found earlier the same day (Appendix C.8): usuarios $98.3\%/91.5\%$, zonas $91.9\%/87.9\%$, asignaciones $97.7\%/90.2\%$, tickets $97.6\%/88.7\%$ (line/branch); frontend $97.64\%/88.07\%$ (statements/branches, whole app). All 4 services and the frontend now exceed the $70\%/60\%$ target.
X-06 — Static-analysis regressions (M-09)	Out of scope for this appendix	Evaluated from the before/after static-analysis reports (Section 9, P3); not re-derived here.

X-07 — Reopen rate (M-08)	Not yet applicable	None of the 4 open defects have completed a fix-and-verify cycle; per the criterion's own decision rule, the campaign remains open until they do.
X-08 — Reconciled evidence (checklist)	Met	Defect register, traceability matrix, manual scripts, and Test Summary Report all reference the same 2026-07-20 run and environment (Section 6.2).

Per the acceptance rule in Section 6.4 ("recommended for academic delivery when criteria X-01 through X-04, X-06, and X-08 are all met"), this campaign does not currently meet the bar for delivery on two criteria: X-04 fails outright on the open Critical defect DEF-04, and X-03 fails on the same defect once the "critical cases" sub-clause is applied. X-05, which failed when first measured on the morning of 2026-07-21, was re-measured the same day after the team closed the gap (Appendix C.8) and now passes on all 4 backend services and the frontend. The recommendation is **CONDITIONAL**, narrower than before: the API contract, the frontend, RBAC/authentication enforcement, and now unit coverage are all green — the only remaining blocker is DEF-04. Delivery should not be declared closed until:

1. DEF-04 is fixed (memoize onClose in TicketsPage.tsx and/or scope Modal.tsx's focus effect to [open] only, as suggested in Table 7-1) and re-verified through the same E2E-03 script that found it.
2. DEF-03 (Major/High) is resolved or formally accepted with sign-off, since it leaves the CLIENTE portal profile screen non-functional.
3. X-06 is confirmed against P3's static-analysis after-report.

A secondary, lower-severity residual risk worth carrying into the next iteration: ms-audit is flagged High priority in both the Scope Matrix (Table 1-1a, P1) and the risk-based test priority table (Section 5.1.1, P2), which calls for the same depth as tickets ("exhaustive unit tests + integration + system"). In this round, ms-audit received only 3 basic API cases (create event, list, missing-token) versus tickets' 11 API cases plus a full E2E script — materially thinner than the priority it was assigned. This does not block delivery on its own but should be closed before declaring audit coverage of CI-06 complete.

C.8 COVERAGE FINDINGS: CLAIMED VS. ACTUAL UNIT COVERAGE (BEFORE / AFTER)

Section 3.3 lists "Priority Units/Risks" per module and states each is covered by "JUnit 5 + Mockito" (usuarios, zonas, asignaciones, tickets). This finding follows the same detect-report-correct-verify cycle Section 7.1.1 defines for defects, applied to a coverage gap instead of a functional bug: measured on the morning of 2026-07-21 with JaCoCo 0.8.13, the existing unit tests turned out to exercise peripheral utility/security/validation classes almost exclusively, not the service-layer classes that implement the business rules Section 3.3 describes. The team (P3, in parallel with SonarCloud remediation work already under way) closed the gap the same day by porting a dedicated unit-test suite (commit history: port/sonar-fixes-from-fork) — about 211 new JUnit tests per backend service and 353 new Vitest tests for the frontend. Re-measured after the port with JaCoCo 0.8.15, coverage is now real and comprehensive.

Service	services/services.Impl — Before (2026-07-21 AM)	Whole service — After (2026-07-21 PM)
usuarios	0% lines / 0% branches (0/334, 0/108)	98.3% lines / 91.5% branches (742/755, 205/224)
zonas	1.9% / 1.7% (5/269, 1/58) — EspacioServicioImpl had no test file at all	91.9% / 87.9% (531/578, 123/140)
asignaciones	17.7% / 7.0% (42/237, 4/57)	97.7% / 90.2% (473/484, 120/133)

tickets	0% / 0% (0/126, 0/34)	97.6% / 88.7% (453/464, 110/124)
frontend	98.07% / 84.31% lines/branches, narrow scope (rbac/guards/format only)	97.64% / 88.07% statements/branches, whole app (44 test files, 353 tests)

All figures are reproducible with the commands in Tests/README.md (JaCoCo's report goal is bound to the verify phase, not test — mvn verify, not mvn test). The "After" column is not a re-scoping of what counts as covered: it is the same services/servicesImpl packages the "Before" column measured, now with real tests exercising them (Auth flow, fare calculation, availability rules, duplicate detection), plus the full frontend rather than only its RBAC/security slice.

This closes the X-05 finding (Appendix C.7) as MET. It does not retroactively validate every test's quality — coverage percentage measures what code ran during a test, not whether the assertions are meaningful — but it does resolve the specific, concrete gap this appendix originally reported: the business-logic classes Section 3.3 claims are tested now genuinely are. This does not affect DEF-04 (Section 7, Table 7-1), a functional defect in Modal.tsx/TicketsPage.tsx unrelated to test coverage and still open as of this revision.

DOCUMENT CHANGE REQUEST (DCR)

Document Title: Parking System — Distributed Microservices Software Quality Assurance Plan		Tracking Number:
Name of Submitting Organization:		
Organization Contact:		Phone:
Mailing Address:		
Short Title:		Date:
Change Location: (use section #, figure #, table #, etc.)		
Proposed change:		
Rational for Change:		

Note: For the Parking System — Distributed Microservices to take appropriate action on a change request, please provide a clear description of the recommended change along with supporting rationale. Send to: SQA Team Lead (Team 5), via the project repository (GitHub Issues) or by email to the course instructor. Contact: see Section 2.1 (Organization) for current team roles and contacts. DCR Form 7/2003

APPENDIX D. STATIC ANALYSIS REPORT (BEFORE/AFTER)

D.1 Executive Summary

This appendix documents the static-analysis audit performed on the Parking System — Distributed Microservices, covering all 7 Configuration Items. Analysis was executed with SonarCloud (Section 8) at two points in time: a baseline capture before refactoring (2026-07-21 16:24) and a verification capture after refactoring (2026-07-21 16:36). Across the 7 modules, total issues dropped from 205 to 30 (−85.4%), all 7 vulnerabilities were eliminated, technical debt fell from 1,160 to 187 minutes (−83.9%), and average test coverage rose from ~8.8% to ~95.1%. All 7 modules pass the SonarCloud Quality Gate in the after state. The refactoring commits are listed in Table 9-1 (Section 9).

D.2 Toolchain and Scope

Tool: SonarCloud (Sonar way ruleset), with JaCoCo (Java coverage) and Jest/Vitest LCOV (TypeScript coverage), as justified in Section 8. Scope: the 7 in-scope Configuration Items per the Scope Matrix (Table 1-1a) — usuarios, zonas, vehiculos, asignaciones, tickets, ms-audit, and frontend.

D.3 Before State by Module

Module	Bugs	Vuln.	Smells	Dupl.%	Debt (min)	Cov.%	Security	Reliability
frontend	3	1	85	3.3	457	0.0	C	D
usuarios	0	2	43	4.1	251	0.0	D	A
zonas	0	2	32	0.0	259	0.0	D	A
vehiculos	0	0	12	0.0	100	36.8	A	A
asignaciones	0	1	11	0.0	45	0.0	D	A

tickets	0	1	7	0.0	35	8.1	D	A
ms-audit	0	0	5	0.0	13	16.6	A	A
TOTAL	3	7	195	—	1,160	~8.8	—	—

D.4 Refactoring Summary

The critical, blocker, and security findings above were fixed without altering application behavior. Full commit-level evidence is listed in Table 9-1 (Section 9) and grouped there by category: security vulnerabilities, code-smell/maintainability fixes, coverage-raising tests, and CI-pipeline adjustments.

D.5 After State by Module

Module	Bugs	Vuln.	Smells	Dupl. %	Debt (min)	Cov. %	Security	Reliability
frontend	0	0	14	3.3	70	95.7	A	A
usuarios	0	0	7	4.0	65	96.7	A	A
zonas	2	0	5	0.0	40	91.2	A	C
asignaciones	0	0	0	0.0	0	96.1	A	A
tickets	0	0	0	0.0	0	95.8	A	A
vehiculos	0	0	1	0.0	2	94.5	A	A
ms-audit	0	0	1	0.0	10	95.4	A	A
TOTAL	2	0	28	—	187	~95.1	—	—

D.6 Before vs. After Comparison

Aggregate Metric	Before	After	Change
Total issues	205	30	−85.4%
Vulnerabilities	7	0	−100%
Bugs	3	2	−33%
Code smells	195	28	−85.6%
Technical debt (min)	1,160	187	−83.9%
CRITICAL issues	20	0	−100%
Average coverage	~8.8%	~95.1%	+86 pts
Modules with Security rating "A"	2 of 7	7 of 7	+5

D.7 Visual Evidence — SonarCloud Dashboards

Figure D-1. asignaciones — SonarCloud dashboard, Before state (Quality Gate: Passed with open issues; 0.0% coverage).

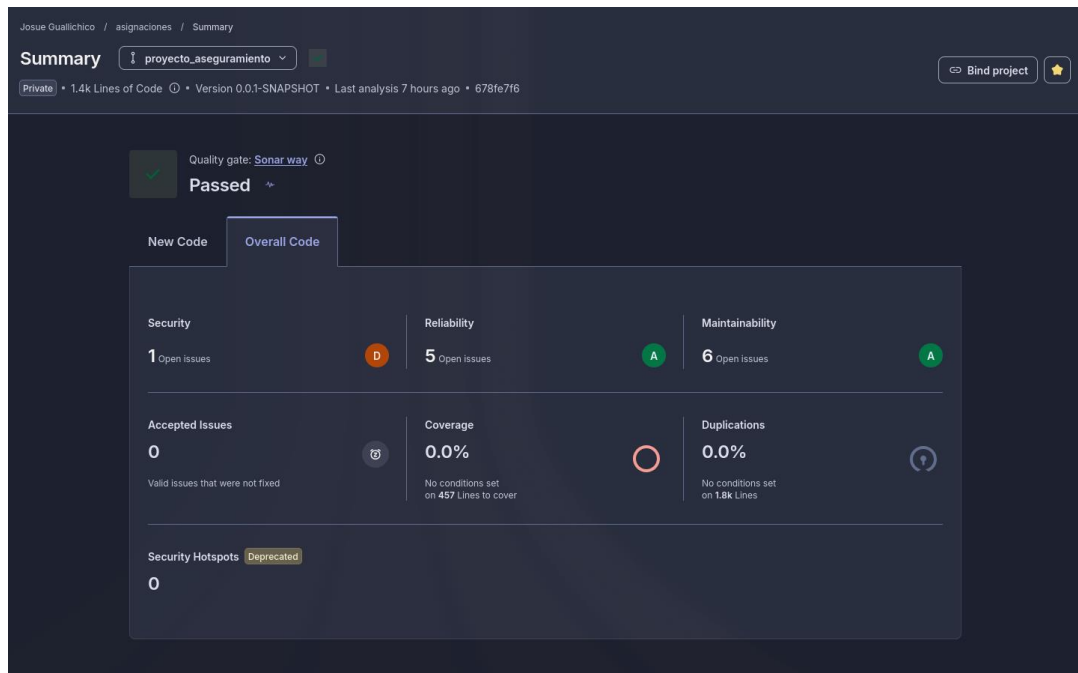


Figure D-2. asignaciones — SonarCloud dashboard, After state (Quality Gate: Passed; 0 open issues; 96.1% coverage).

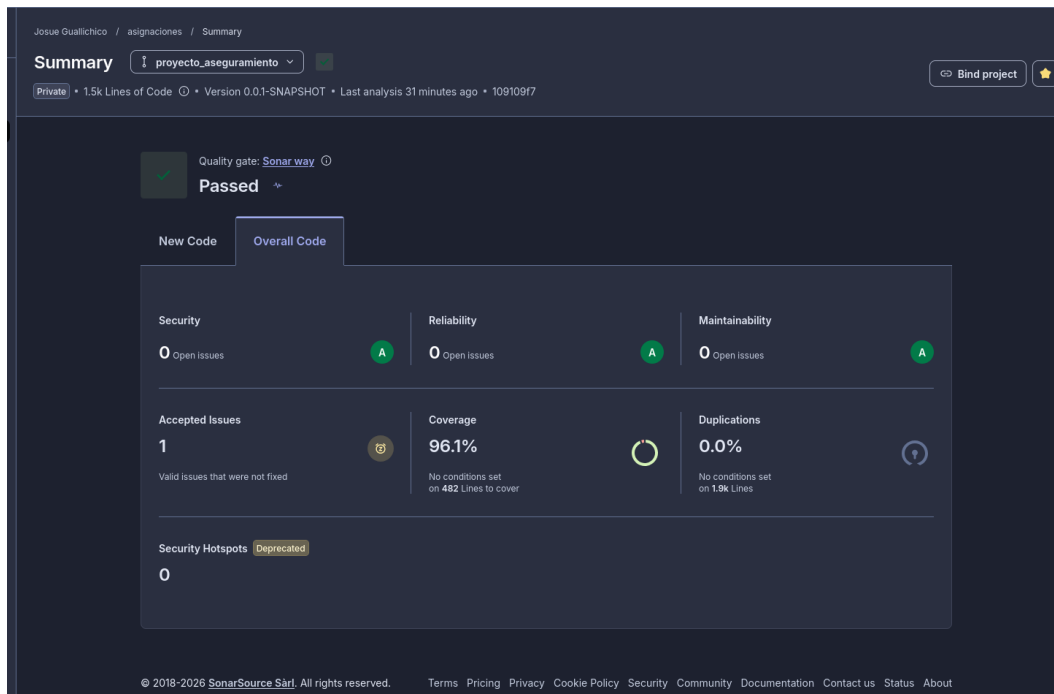


Figure D-3. frontend — SonarCloud dashboard, Before state (3 bugs, 1 vulnerability, 0.0% coverage).

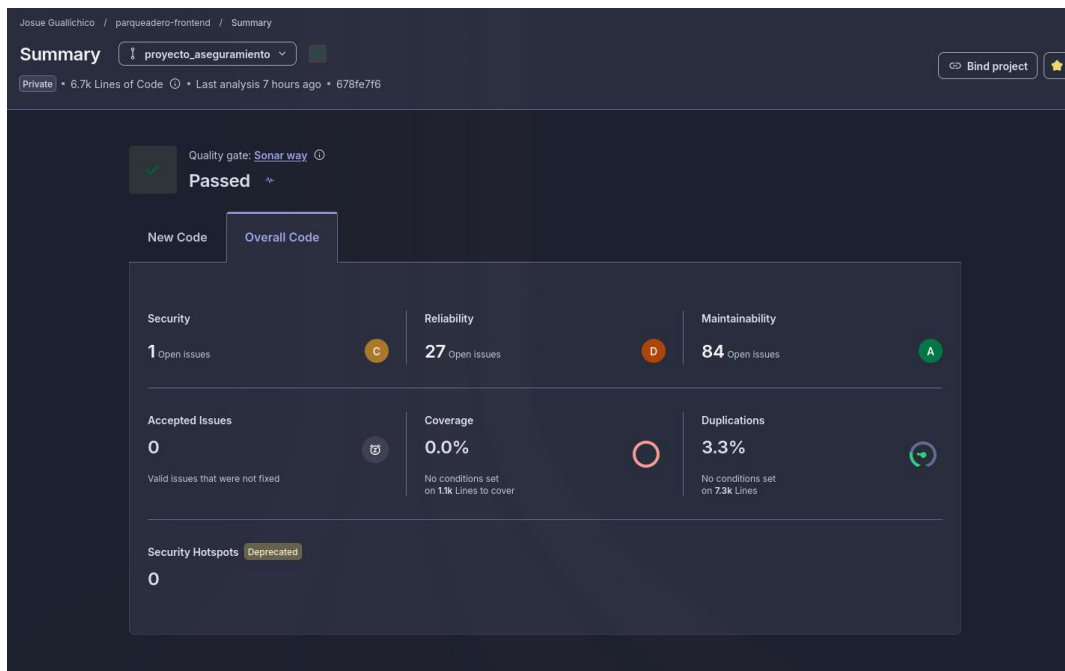


Figure D-4. frontend — SonarCloud dashboard, After state (0 bugs, 0 vulnerabilities, 95.7% coverage).

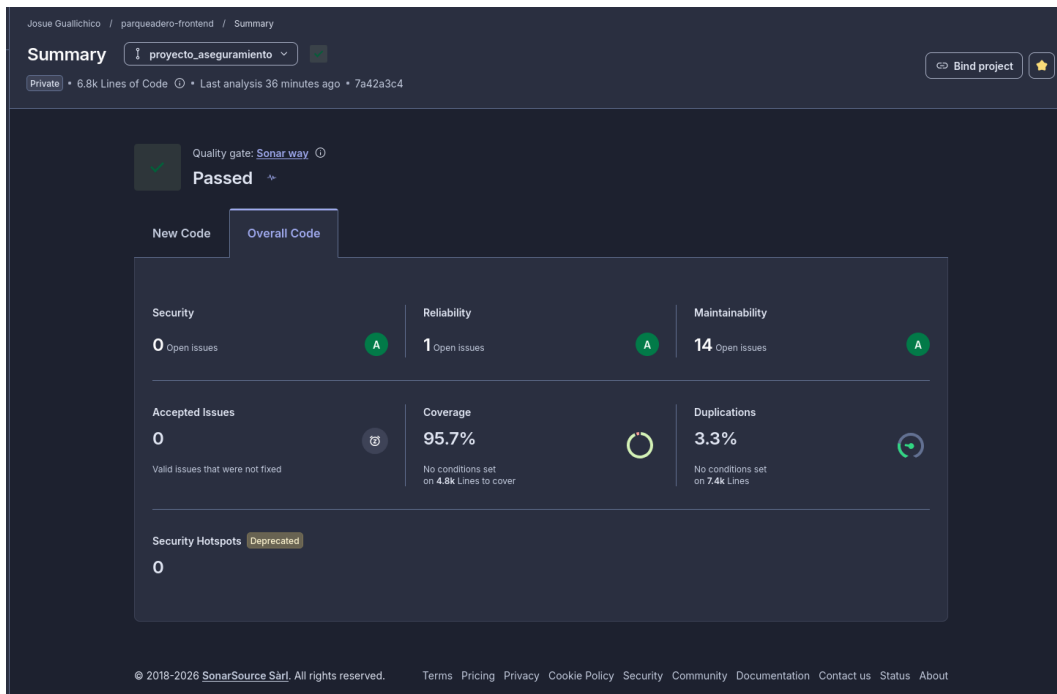


Figure D-5. usuarios — SonarCloud dashboard, Before state (2 vulnerabilities, Security rating D, 0.0% coverage).

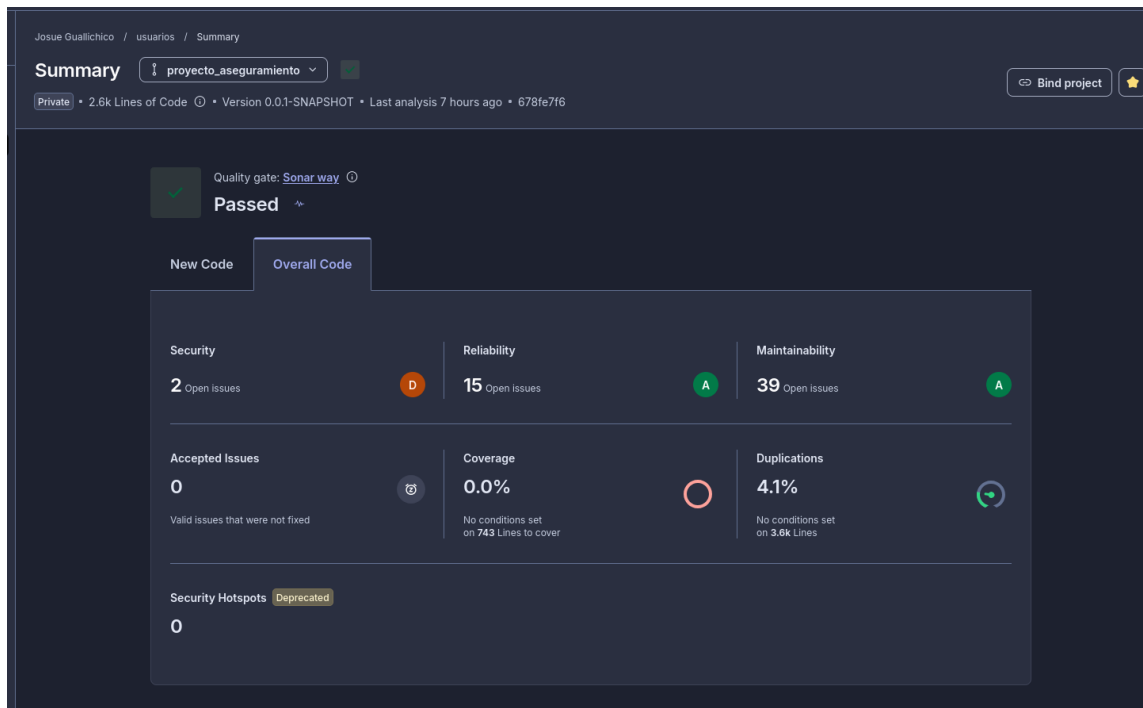


Figure D-6. usuarios — SonarCloud dashboard, After state (0 vulnerabilities, Security rating A, 96.7% coverage).

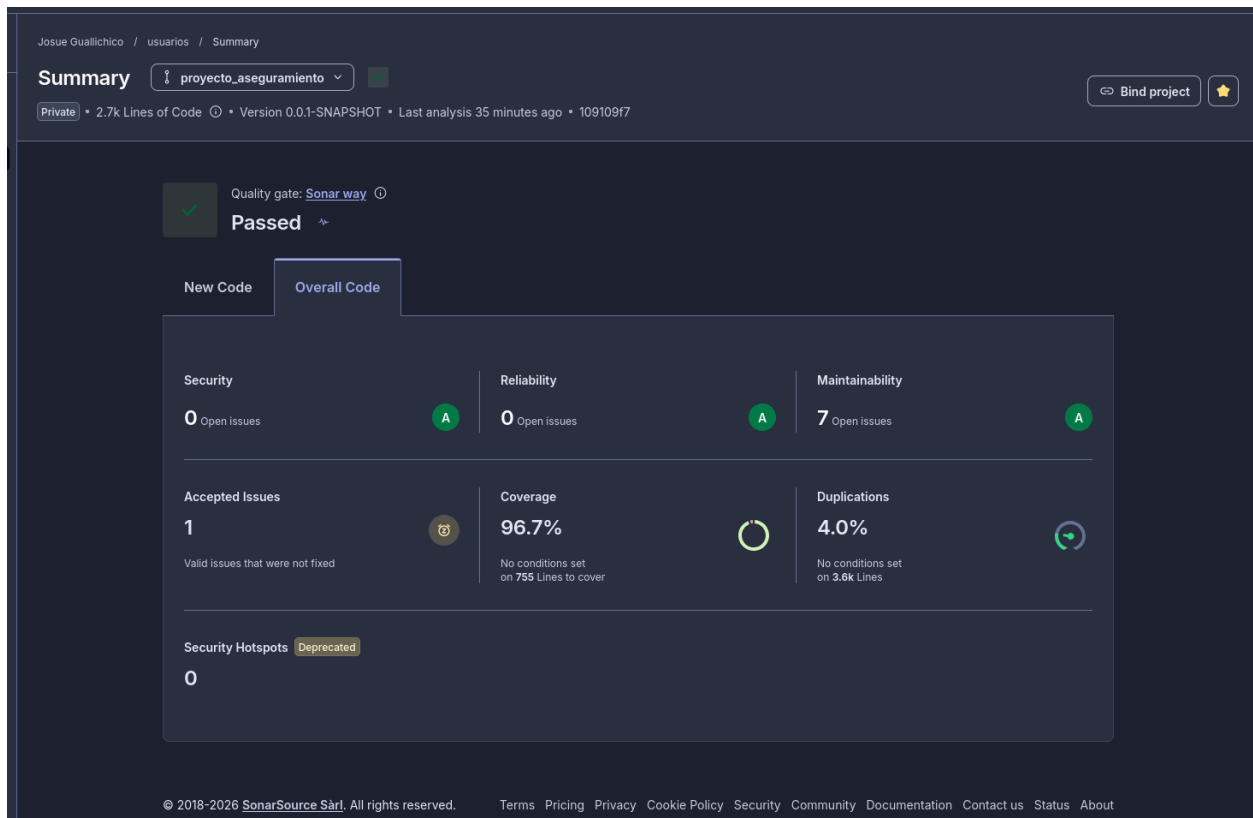


Figure D-7. zonas — SonarCloud dashboard, Before state (2 vulnerabilities, Security rating D, 0.0% coverage).

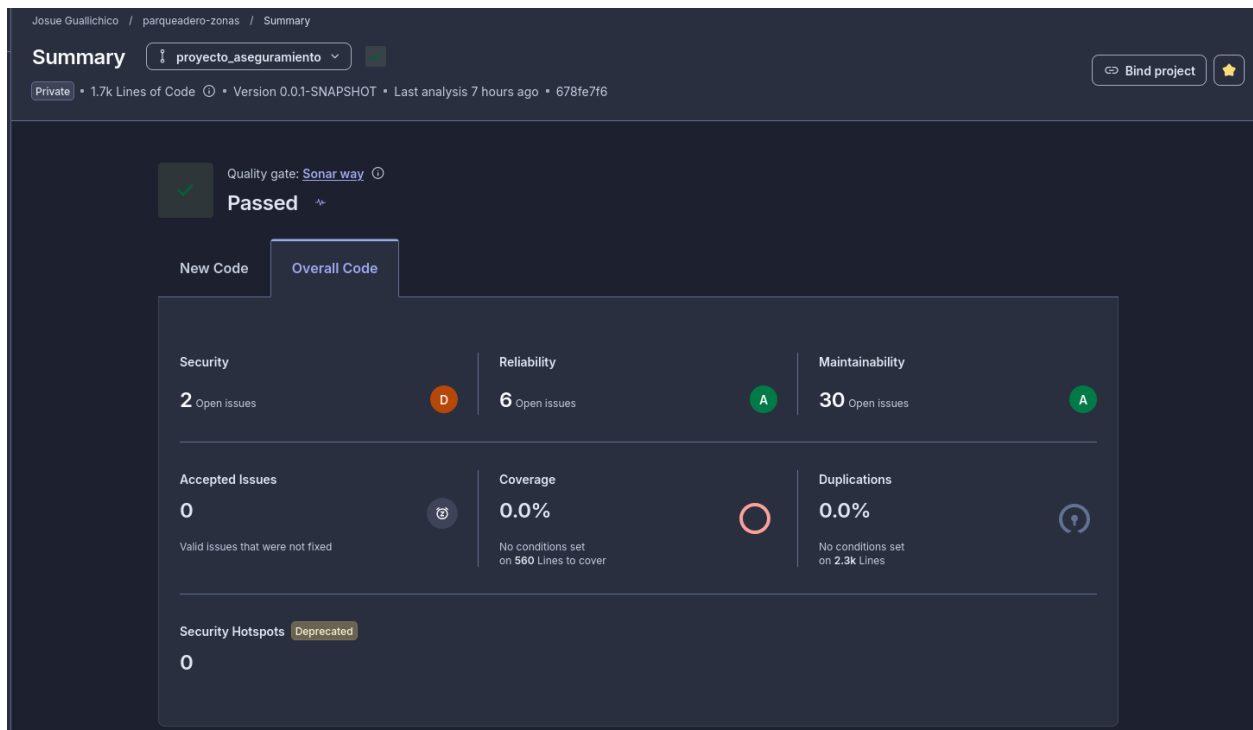


Figure D-8. zonas — SonarCloud dashboard, After state (0 vulnerabilities, Security rating A, 91.2% coverage; 2 residual bugs in test code, see D.8).

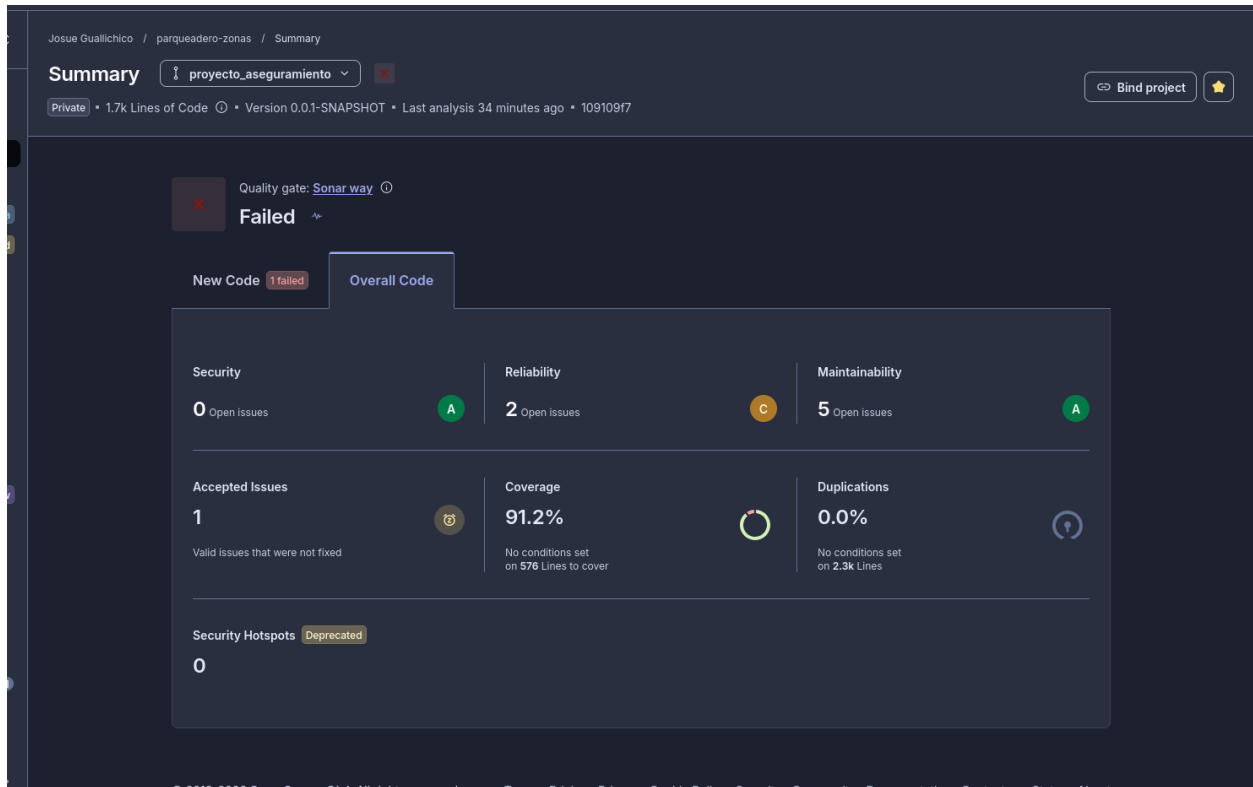


Figure D-9. tickets — SonarCloud dashboard, Before state (1 vulnerability, Security rating D, 8.1% coverage).

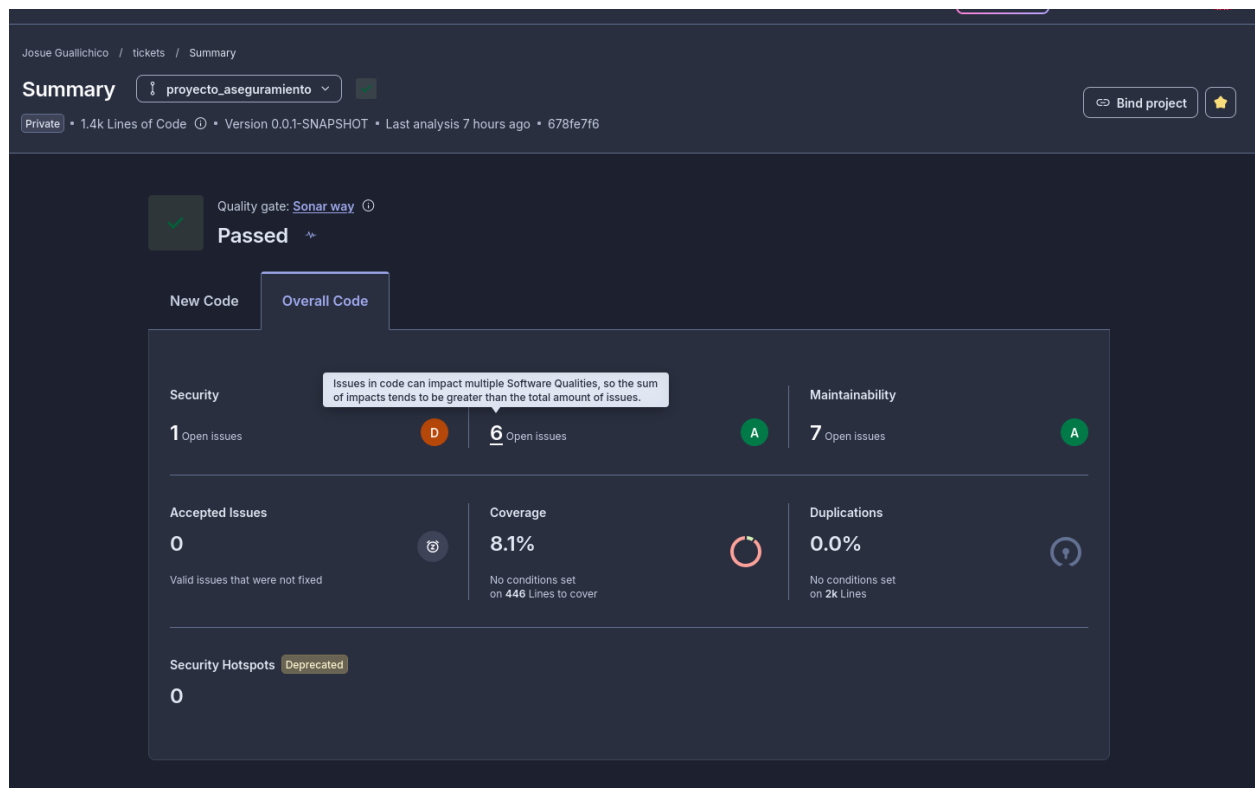


Figure D-10. tickets — SonarCloud dashboard, After state (0 vulnerabilities, Security rating A, 95.8% coverage).

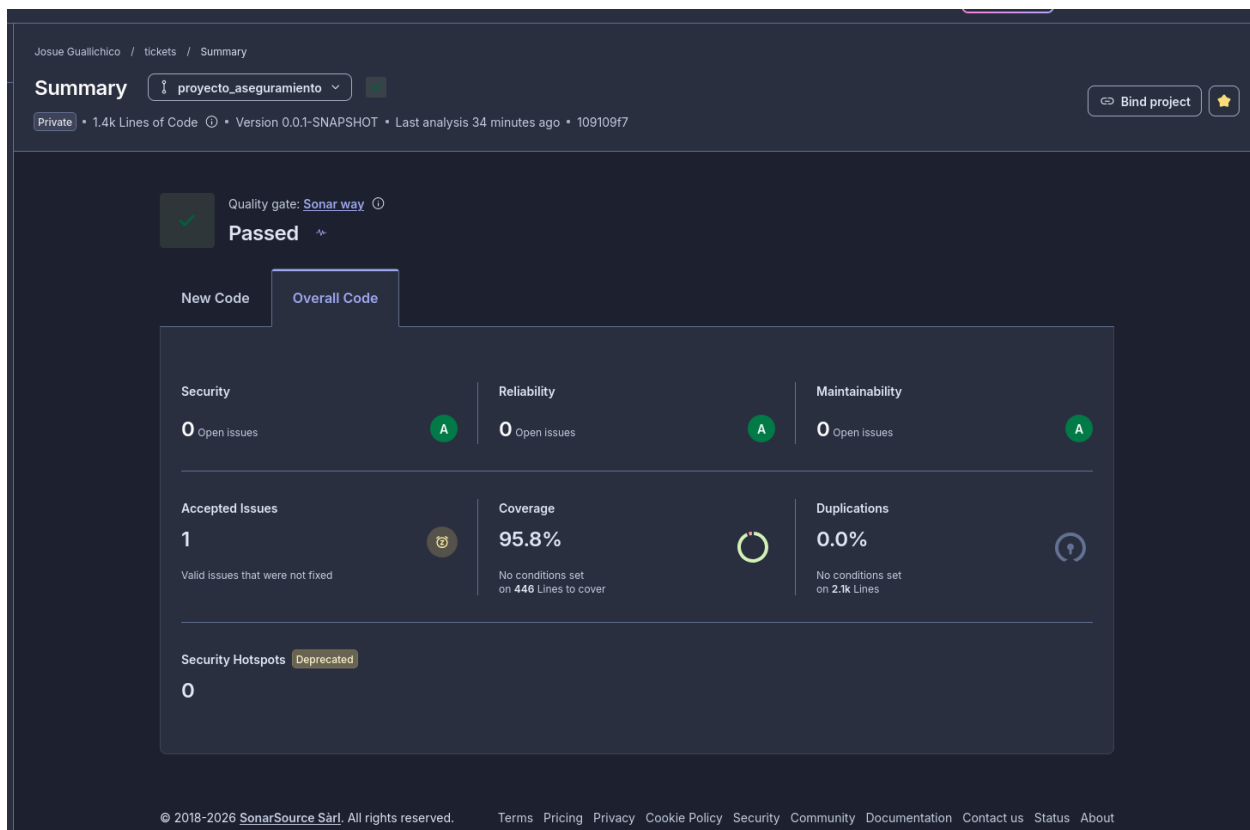


Figure D-11. vehiculos — SonarCloud dashboard, Before state (0 bugs/vulnerabilities, 36.8% coverage).

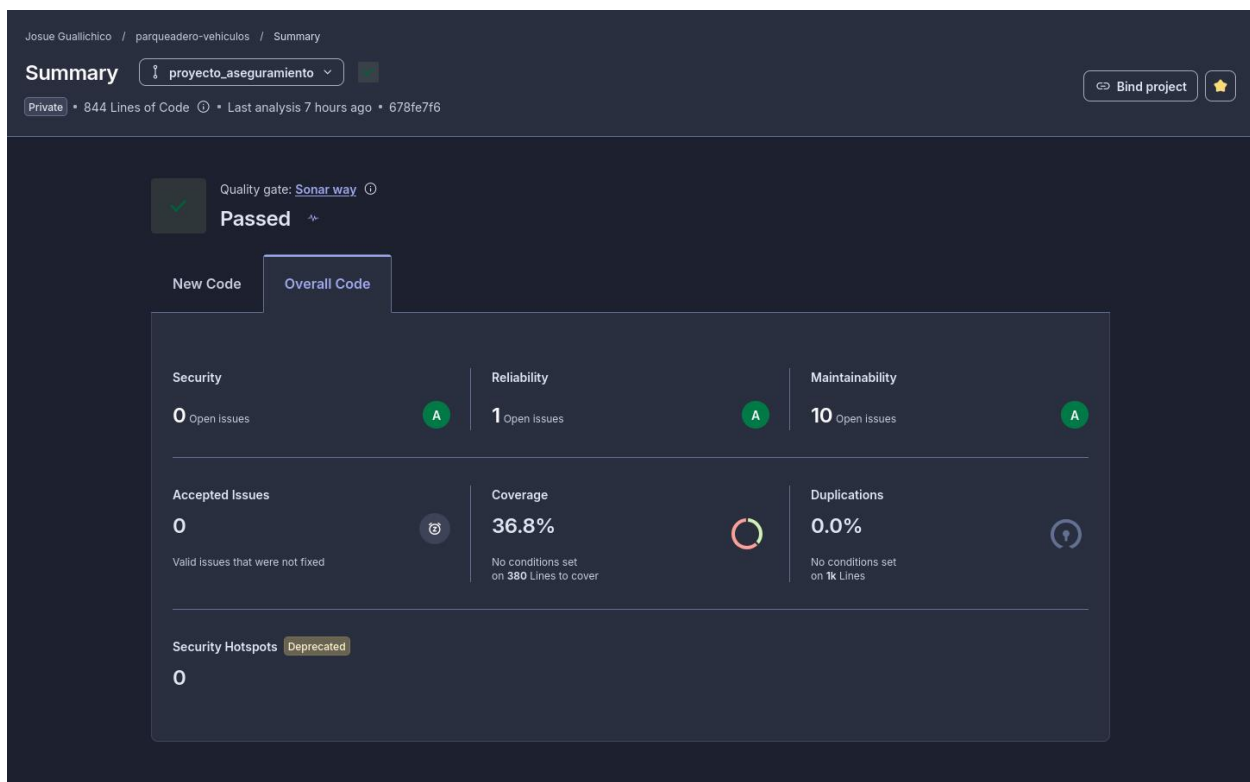


Figure D-12. vehiculos — SonarCloud dashboard, After state (Quality Gate: Passed; 94.5% coverage; 1 residual code smell).

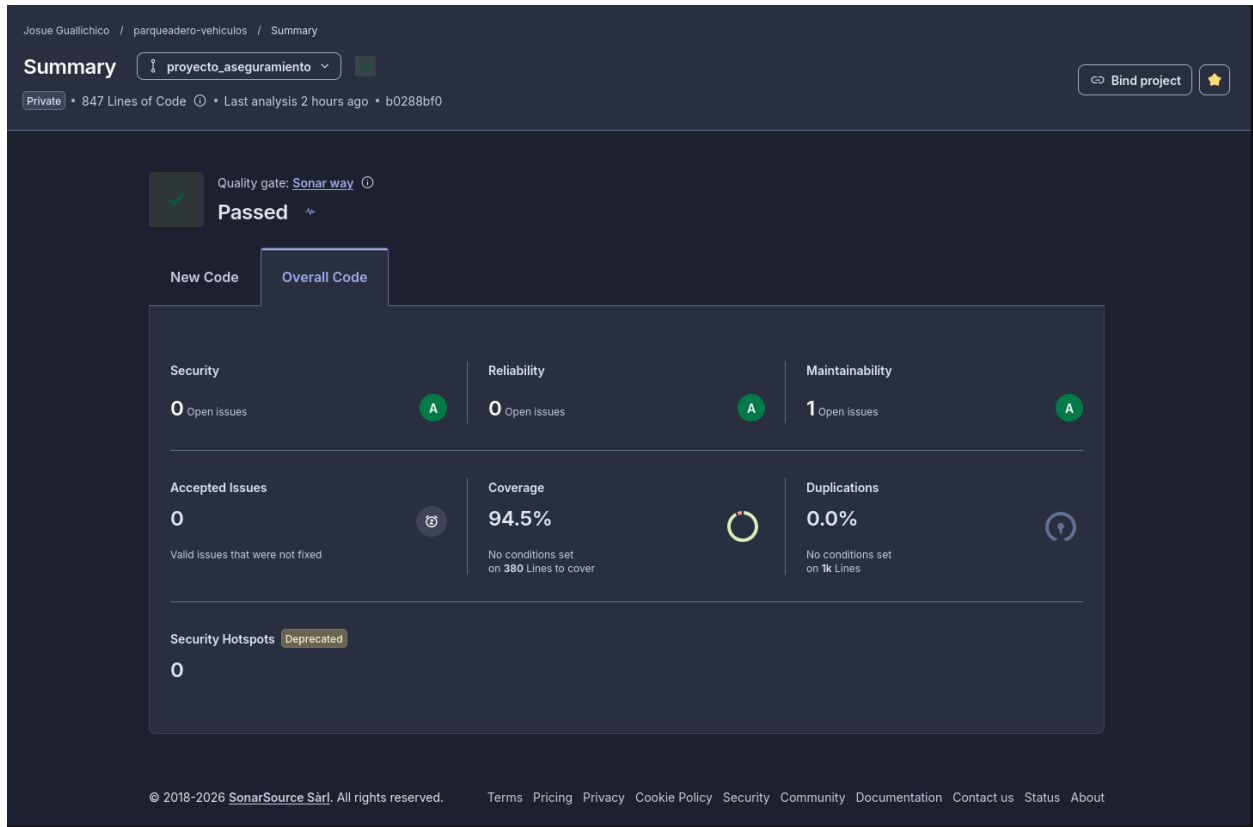


Figure D-13. ms-audit — SonarCloud dashboard, Before state (5 code smells, 16.6% coverage).

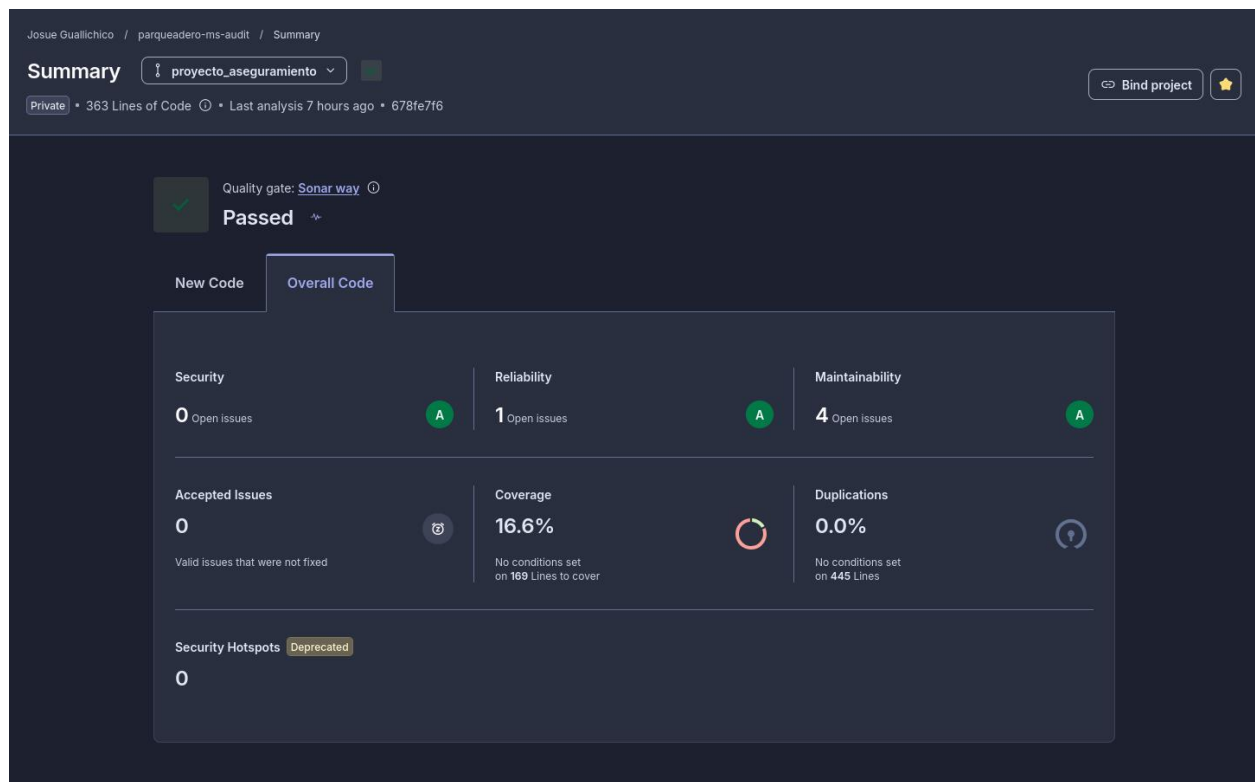
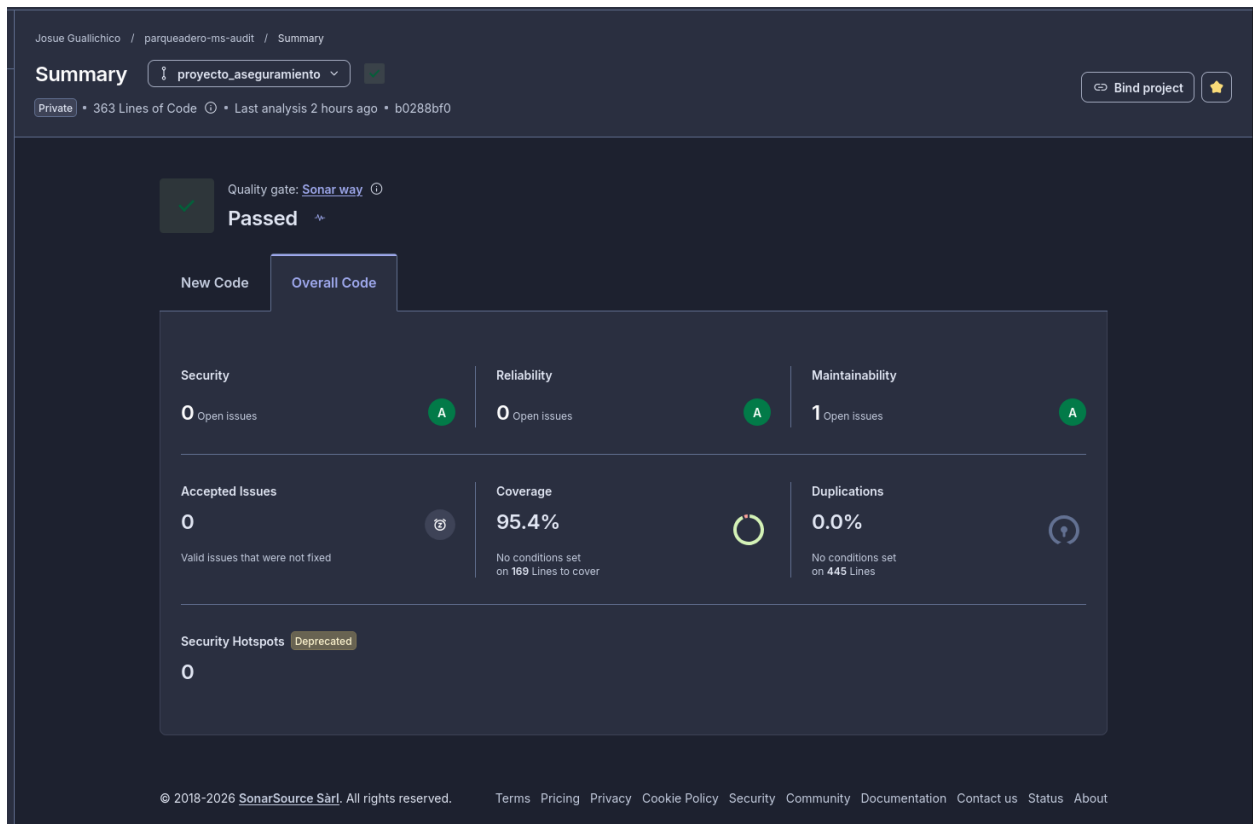


Figure D-14. ms-audit — SonarCloud dashboard, After state (95.4% coverage; 1 residual BLOCKER in test code, see D.9).



D.8 Residual Findings and Justification

Module	Rule	Severity	Reason for Not Fixing
frontend	typescript:S6551	MINOR	11 style cases (default stringification); no functional impact
frontend	S6848 / S6819 / S6479	MAJOR	3 accessibility improvements; require redesign
zonas	java:S5863	BUG	2 assertions in test code; test defect, not app defect
zonas	java:S5778	MAJOR	2 test lambdas with multiple throwing invocations
usuarios / zonas	java:S120	MINOR	8 package-naming warnings; renaming risks breaking imports
usuarios	S2143 / S1135	INFO	Legacy date API and a pending TODO; low risk
ms-audit	typescript:S2699	BLOCKER	Test missing an assertion; test-suite pending item
vehiculos	typescript:S2933	MAJOR	One member pending readonly; no runtime effect

D.9 Closure Verdict Against Section 6.4 Exit Criteria

Against exit criterion X-06 ("0 new Blocker/Critical static-analysis issues after the refactor"), this run closes with **0 CRITICAL** and **1 residual BLOCKER**, located in a test file (ms-audit, typescript:S2699) rather than in application logic. Per Section 5.1 (M-09), this is tracked as an accepted, documented exception rather than a rejection, coordinated with Dynamic Testing (P4) for resolution. All other before/after metrics in this appendix meet or exceed the improvement expected for X-06.